

CANARA ENGINEERING COLLEGE
Benjanapadavu, Mangalore - 574219



LABORATORY MANUAL

DEVOPS
(BCSL657D)

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

(Accredited by National Board of Accreditation, New Delhi.)

VISION

To be recognized as a center of knowledge dissemination in Computer Science and Engineering by imparting value-added education to transform budding minds into competent computer professionals.

MISSION

- Provide a value-based learning environment enriched with ethics, honesty and integrity that equips students to cater to the needs of society and industry.
- Augment the knowledge of students towards cutting-edge technologies and state-of-the-art tools of Computer Science & Engineering.
- Create opportunities for all-round development of students through co-curricular and extra-curricular activities.
- Promote research, innovation and development activities in the field of Computer Science among staff and students without any bias.

PROGRAMME EDUCATIONAL OBJECTIVES

1. Graduates will work productively as computer science engineers exhibiting ethical qualities and leadership roles in multi-disciplinary teams.
2. Graduates will adapt to the changing technologies, tools and societal requirements.
3. Graduates will design and deploy software that meets the needs of individuals and the industries.
4. Graduates will take up higher education and/or be associated with the field so that they can keep themselves abreast of Research & Development.

PROGRAMME OUTCOMES

Engineering graduates in Computer Science and Engineering will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and

design system components or processes that meet the specific needs with appropriate consideration for the public health and safety, and the cultural, societal and environmental considerations.

4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods, including design of experiments, analysis and interpretation of data and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Select/Create and apply appropriate techniques, resources and modern engineering and IT tools, including prediction and modeling to complex engineering activities, taking comprehensive cognizance of their limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and Sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the relevant scientific and/or engineering practices.
9. **Individual and team work:** Function effectively as an individual and as a member or leader in diverse teams and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with the society-at-large, such as being able to comprehend and write effective reports and design documentation, make effective presentations and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work as a member and leader in a team to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for and above have the preparation and ability to engage in independent and life-long learning in the broadcast context of technological changes.

DEVOPS		Semester	6
Course Code	BCSL657D	CIE Marks	50
Teaching Hours/Week (L:T:P: S)	0:0:2:0	SEE Marks	50
Credits	01	Exam Hours	100
Examination type (SEE)	Practical		
Course objectives: • To introduce DevOps terminology, definition & concepts • To understand the different Version control tools like Git, Mercurial • To understand the concepts of Continuous Integration/ Continuous Testing/ Continuous Deployment) • To understand Configuration management using Ansible • Illustrate the benefits and drive the adoption of cloud-based Devops tools to solve real world problems			
Sl.NO	Experiments		
1	Introduction to Maven and Gradle: Overview of Build Automation Tools, Key Differences Between Maven and Gradle, Installation and Setup		
2	Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins		
3	Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation		
4	Practical Exercise: Build and Run a Java Application with Maven, Migrate the Same Application to Gradle		
5	Introduction to Jenkins: What is Jenkins?, Installing Jenkins on Local or Cloud Environment, Configuring Jenkins for First Use		
6	Continuous Integration with Jenkins: Setting Up a CI Pipeline, Integrating Jenkins with Maven/Gradle, Running Automated Builds and Tests		
7	Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks, and Modules, Automating Server Configurations with Playbooks, Hands-On: Writing and Running a Basic Playbook		
8	Practical Exercise: Set Up a Jenkins CI Pipeline for a Maven Project, Use Ansible to Deploy Artifacts Generated by Jenkins		
9	Introduction to Azure DevOps: Overview of Azure DevOps Services, Setting Up an Azure DevOps Account and Project		
10	Creating Build Pipelines: Building a Maven/Gradle Project with Azure Pipelines, Integrating Code Repositories (e.g., GitHub, Azure Repos), Running Unit Tests and Generating Reports		
11	Creating Release Pipelines: Deploying Applications to Azure App Services, Managing Secrets and Configuration with Azure Key Vault, Hands-On: Continuous Deployment with Azure Pipelines		
12	Practical Exercise and Wrap-Up: Build and Deploy a Complete DevOps Pipeline, Discussion on Best Practices and Q&A		

Course outcomes (Course Skill Set):

At the end of the course the student will be able to:

- Demonstrate different actions performed through Version control tools like Git
- Perform Continuous Integration and Continuous Testing and Continuous Deployment using Jenkins by building and automating test cases using Maven & Gradle.
- Experiment with configuration management using Ansible.
- Demonstrate Cloud-based DevOps tools using Azure DevOps.
- To build and deploy a complete DevOps pipeline, integrating tools like Jenkins, Maven/Gradle, Ansible, and Azure DevOps, while understanding best practices in DevOps workflows.

Assessment Details (both CIE and SEE)

The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together

Continuous Internal Evaluation (CIE):

CIE marks for the practical course are **50 Marks**.

The split-up of CIE marks for record/ journal and test are in the ratio **60:40**.

- Each experiment is to be evaluated for conduction with an observation sheet and record write-up. Rubrics for the evaluation of the journal/write-up for hardware/software experiments are designed by the faculty who is handling the laboratory session and are made known to students at the beginning of the practical session.
- Record should contain all the specified experiments in the syllabus and each experiment write-up will be evaluated for 10 marks.
- Total marks scored by the students are scaled down to **30 marks** (60% of maximum marks).
- Weightage to be given for neatness and submission of record/write-up on time.
- Department shall conduct a test of 100 marks after the completion of all the experiments listed in the syllabus.
- In a test, test write-up, conduction of experiment, acceptable result, and procedural knowledge will carry a weightage of 60% and the rest 40% for viva-voce.
- The suitable rubrics can be designed to evaluate each student's performance and learning ability.
- The marks scored shall be scaled down to **20 marks** (40% of the maximum marks).

The Sum of scaled-down marks scored in the report write-up/journal and marks of a test is the total CIE marks scored by the student.

Semester End Evaluation (SEE):

- SEE marks for the practical course are 50 Marks.
- SEE shall be conducted jointly by the two examiners of the same institute, examiners are appointed by the Head of the Institute.

- The examination schedule and names of examiners are informed to the university before the conduction of the examination. These practical examinations are to be conducted between the schedule mentioned in the academic calendar of the University.
- All laboratory experiments are to be included for practical examination.
- (Rubrics) Breakup of marks and the instructions printed on the cover page of the answer script to be strictly adhered to by the examiners. **OR** based on the course requirement evaluation rubrics shall be decided jointly by examiners.

What is DevOps?

DevOps is a set of cultural philosophies, practices, and tools that combine software development (Dev) and IT operations (Ops). Its goal is to shorten the systems development life cycle while delivering features, fixes, and updates frequently in close alignment with business objectives.

DevOps promotes:

- **Collaboration:** Breaking down silos between development and operations teams.
- **Automation:** Automating repetitive tasks (builds, tests, deployments) to improve efficiency.
- **Continuous Integration/Delivery (CI/CD):** Regularly integrating code changes and deploying them to production with minimal manual intervention.
- **Monitoring and Feedback:** Constantly monitoring the performance and behavior of applications in production to rapidly address issues.

Why is DevOps Used?

- **Speed and Agility:** Faster development cycles and quicker time-to-market.
- **Quality:** Automated testing and integration help catch issues early.
- **Reliability:** Frequent, smaller updates reduce the risk of large-scale failures.
- **Efficiency:** Automation reduces manual errors and repetitive tasks.
- **Scalability:** Processes and infrastructure can grow with the business.

Examples of DevOps in Action:

- **Continuous Integration (CI):** Tools like Jenkins or Azure Pipelines automatically build and test code on every commit.
- **Continuous Deployment (CD):** Systems automatically deploy tested code to production environments.
- **Monitoring:** Tools such as Prometheus, Grafana, or New Relic continuously monitor application performance.

Program1: Introduction to Maven and Gradle: Overview of Build Automation Tools, Key Differences Between Maven and Gradle, Installation and Setup

Introduction to Maven and Gradle

What is Maven?

Maven is a build automation and project management tool primarily used for Java projects. It uses a central configuration file known as the **POM (Project Object Model)**, written in XML, which defines the project structure, its dependencies, build order, and plugins.

Key Features and Roles of Maven:

- **Convention over Configuration:** Maven enforces a standard directory structure (e.g., src/main/java, src/test/java), which simplifies project setup.
- **Dependency Management:** Automatically downloads and manages external libraries and dependencies from repositories like Maven Central.
- **Build Lifecycle:** Defines a fixed lifecycle (e.g., compile, test, package, install, deploy) which standardizes the build process.
- **Plugin Ecosystem:** Provides a rich set of plugins to extend functionality (e.g., unit testing, code coverage, reporting).
-

What is Gradle?

Gradle is a modern build automation tool that is known for its flexibility and performance. It uses a Groovy or Kotlin DSL (Domain Specific Language) to define build logic, which allows for more dynamic and customizable configurations compared to Maven's XML-based approach.

Key Features and Roles of Gradle:

- **Flexible Build Scripts:** Instead of a rigid XML file, Gradle build scripts written in Groovy or Kotlin allow you to include logic, conditionals, and loops.
- **Incremental Builds:** Gradle tracks changes in source files and only rebuilds what is necessary, which can significantly speed up the build process.
- **Multi-project Builds:** Easily manages complex projects with multiple modules or subprojects.
- **Extensibility:** A robust plugin system that enables you to integrate various languages, frameworks, and tools.
- **Parallel Execution:** Can run tasks in parallel, optimizing build times for large projects.

Why Use Maven and Gradle?

Both Maven and Gradle are used to automate the build process and manage project dependencies. Here's why they are so popular in the DevOps ecosystem:

- **Automated Build and Testing:** They allow developers to compile code, run tests, and package applications without manual intervention.
- **Consistent Build Environment:** By enforcing standardized project structures and dependency management, they help avoid the "it works on my machine" problem.
- **Ease of Integration:** Both tools integrate well with Continuous Integration (CI) servers like Jenkins, enabling automated pipelines.
- **Dependency Resolution:** They simplify the process of managing external libraries and ensure that all developers are using the same versions.

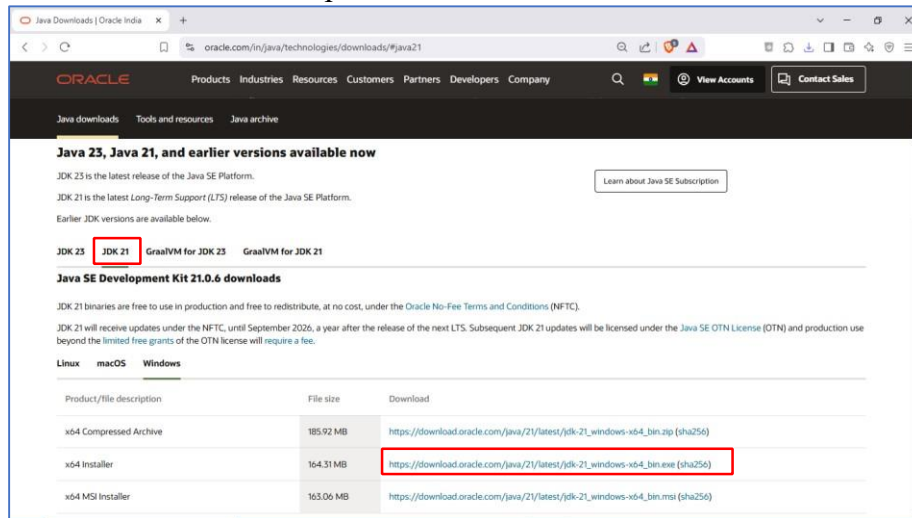
Key Differences Between Maven and Gradle

Aspect	Maven	Gradle
Build Script Language	Uses an XML-based configuration file (pom.xml).	Uses a DSL based on Groovy or Kotlin (build.gradle or build.gradle.kts).
Configuration Style	Declarative and rigid – follows strict conventions (convention over configuration).	Flexible and dynamic – allows you to write custom logic and conditions within the build script.
Build Lifecycle	Provides a fixed lifecycle (e.g., validate, compile, test, package, install, deploy).	Uses a task-based approach where tasks can be defined, customized, and linked in a flexible manner.
Dependency Management	Manages dependencies through the POM file; downloads them from central repositories (e.g., Maven Central).	Similar dependency management; supports dynamic version resolution and customizable dependency configurations.

Performance	Generally slower for large projects because of the fixed build lifecycle and less emphasis on incremental builds.	Often faster, thanks to incremental builds, caching, and parallel execution of tasks.
Extensibility & Plugins	Rich ecosystem of plugins but customization can be more challenging due to XML's verbosity and limitations in scripting.	Highly extensible through its scripting capabilities; writing custom tasks or plugins in Groovy/Kotlin is more straightforward.
Multi-Project Builds	Handles multi-module projects well but requires a strict directory layout and a parent POM for aggregating modules.	Excels in multi-project builds with simple configuration, allowing each subproject to be configured in a flexible manner.
Learning Curve	Easier for beginners due to its structured, convention-based approach but can become complex with large configurations.	More flexible but may have a steeper learning curve initially if you need to leverage its dynamic features and custom logic.
Community & Maturity	Has been around longer, so many legacy projects and extensive documentation exist.	Relatively newer; it has gained popularity due to its modern features, especially in Android and multi-language projects.
Integration with CI/CD	Well-supported by most CI/CD tools (like Jenkins, Azure Pipelines) with stable, predictable behavior.	Also integrates seamlessly with CI/CD tools and is often chosen for its faster build times and flexibility in pipeline scripting.

Java Installation

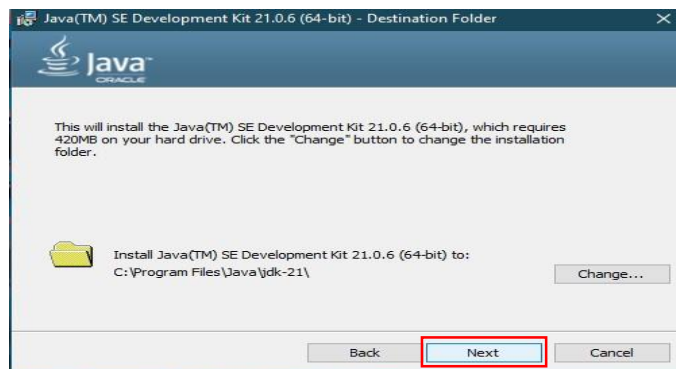
1. Visit the official Oracle website <https://www.oracle.com/in/java/>. Click on the **Windows** option and then **click the x64 Installer** option to download the **.exe file for 64-bit Windows OS**.



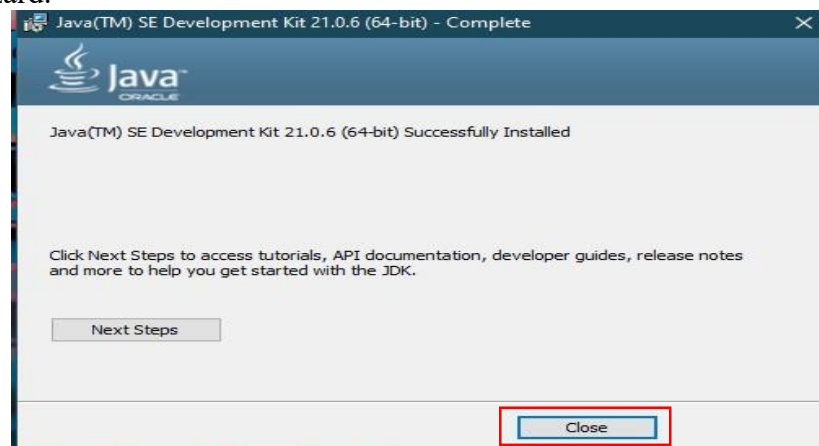
2. Go to **Downloads** and double-click on that downloaded **jdk-21_windows-x64_bin.exe** file. So now **Java Installation Wizard** get open and then click **Next** button.



3. Choose the destination folder for the Java installation files, or stick to the default path and click **Next**.

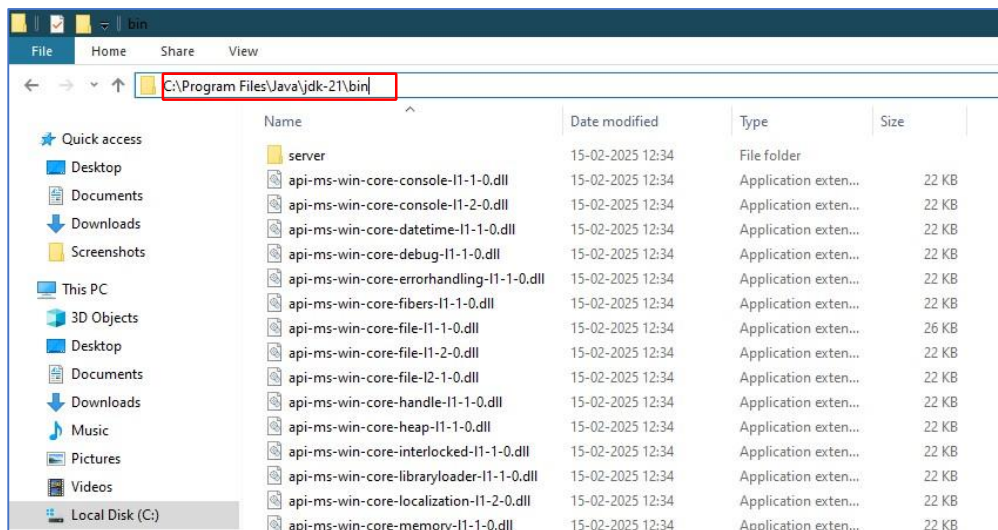


4. The installation process is complete when the Successfully Installed message appears. Click **Close** to exit the wizard.



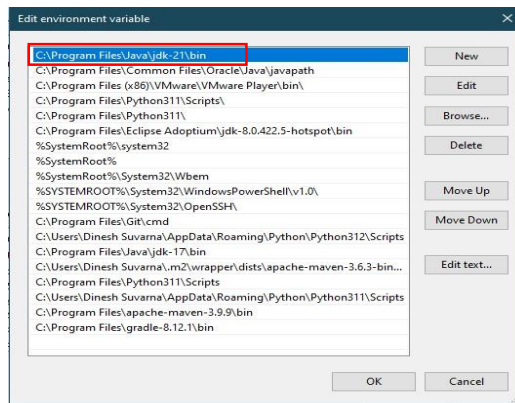
5. set the environment variables

- ✦ Open the C drive, go to Program Files > Java > jdk-2 (or your installed version) > bin folder. Copy the path.



- ✦ Click **Environment Variables**.
- ✦ Select the **Path** variable in the System variables category and click **Edit**.
- ✦ Click **New**.

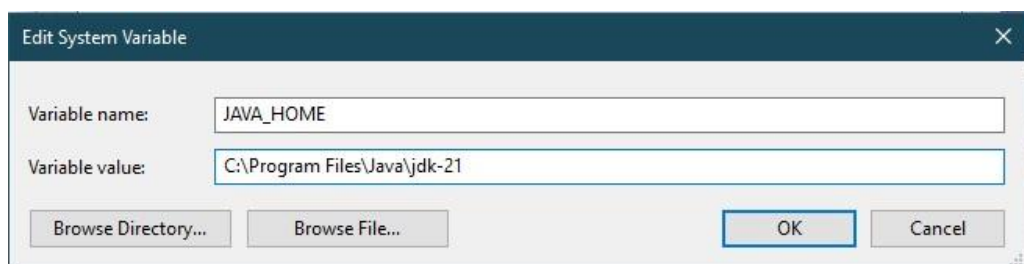
- ✦ Enter the path to the Java **bin** directory.



- ✦ Click **OK** to save the changes and exit the variable editing window.

6. Add **JAVA_HOME** Variable

- ✦ Click **New** under the System variables category to create a new variable. Name the variable **JAVA_HOME**.
- ✦ Enter the path to your Java JDK directory (**C:\Program Files\java\jdk-x**) in the variable value field.



- ✦ Confirm the changes by clicking **OK** in the Environment Variables and System properties windows.

7. Verify the Installation

- ✦ Verify that Java is installed by entering the **java -version** command in the command prompt.

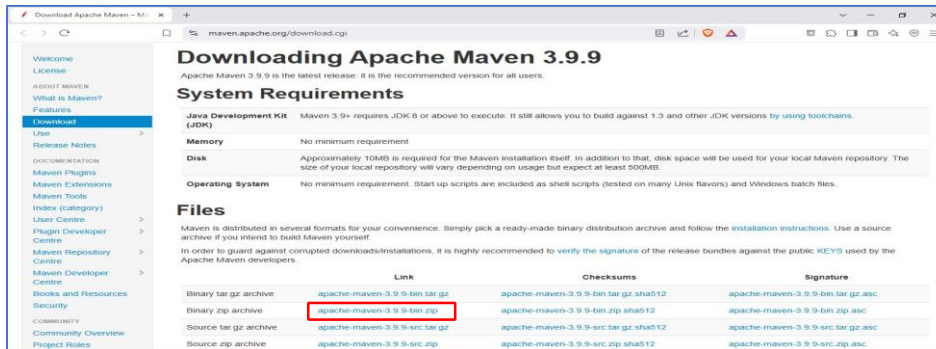
```

C:\Users\Dinesh Suvarna>java --version
java 21.0.6 2025-01-21 LTS
Java(TM) SE Runtime Environment (build 21.0.6+8-LTS-188)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.6+8-LTS-188, mixed mode, sharing)
  
```

Maven Installation

1. Download Maven:

- Go to the <https://maven.apache.org/download.cgi> and download the latest binary ZIP file.



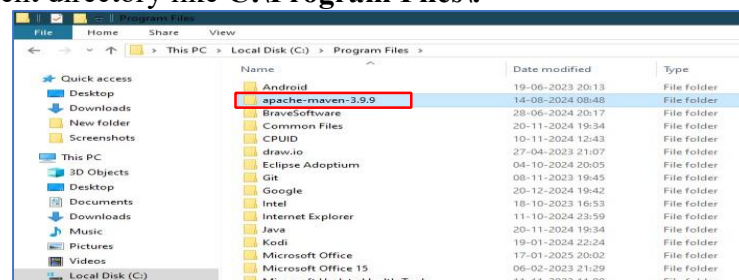
2. Extract the ZIP File:

- Right-click the downloaded ZIP file and select **Extract All**.



3. Move the Folder:

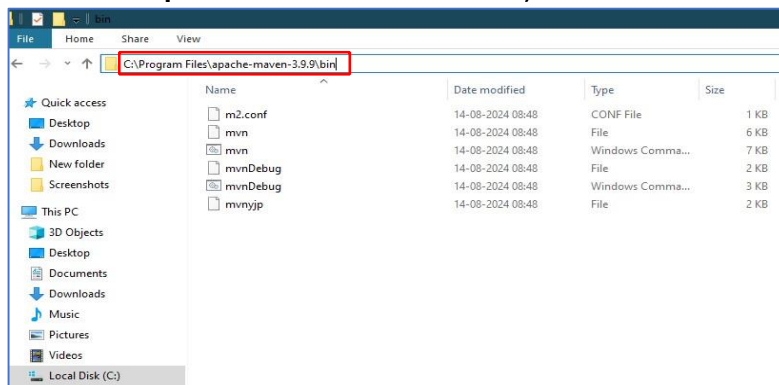
- After extraction, move the extracted **Maven folder (apache-maven-x.x.x)** to a convenient directory like **C:\Program Files**.



4. Navigate to the bin Folder:

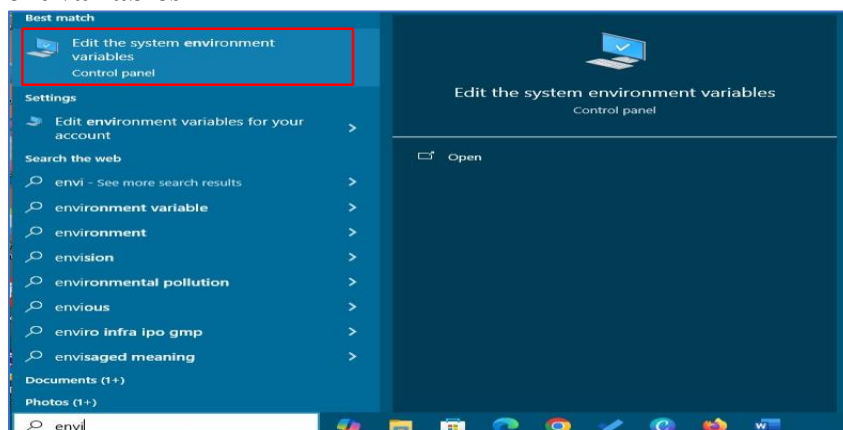
- Open the **Maven folder**, then navigate to the **bin** folder inside.
- Copy the path from the File Explorer address bar (**C:\Program**

Files\apache-maven-x.x.x\bin).

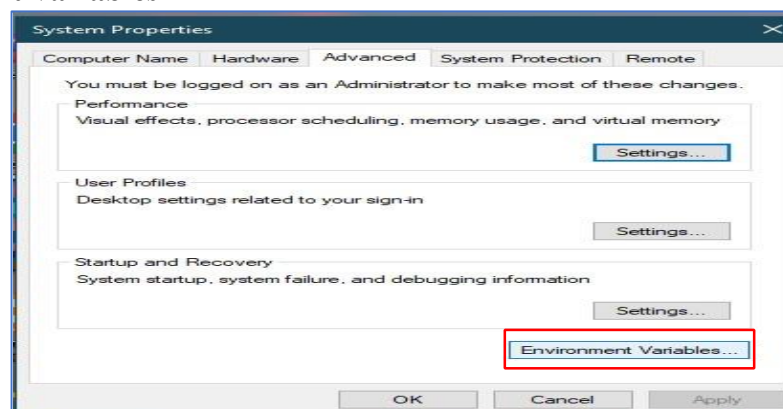


5. Set Environment Variables:

- Open the Start Menu, search for **Environment Variables**, and select **Edit the system environment variables**.

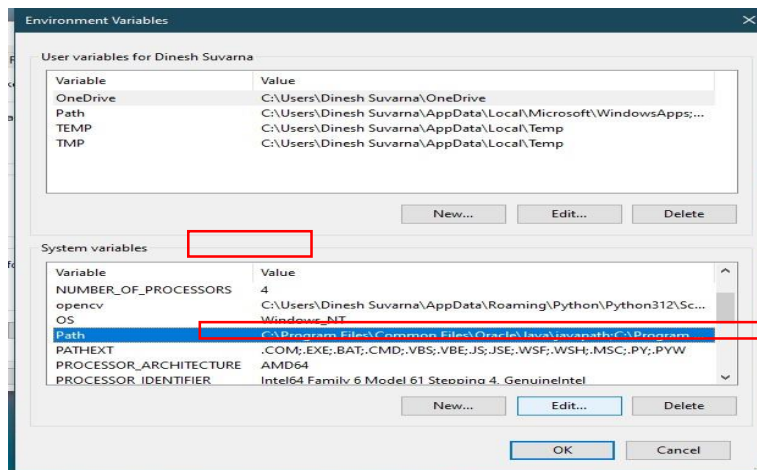


- Click **Environment Variables**.

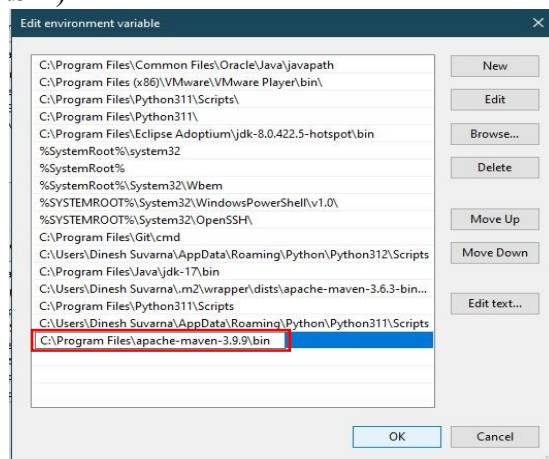


Under **System Variables**:

- Find the **path**, double click on it and click **New**.



- Paste the full path to the bin folder of your Maven directory (e.g., **C:\Program Files\apache-maven-x.x.x\bin**).



6. Save the Changes:

- o Click **OK** to close the windows and save your changes.

7. Verify the Installation:

- Open Command Prompt and run: **mvn -v** If Maven is correctly installed, it will display the version number.

```

C:\Users\Dinesh Suvarna>mvn -v
Apache Maven 3.9.9 (8e8579a9e76f7d015ee5ec7bfcdc97d260186937)
Maven home: C:\Program Files\apache-maven-3.9.9
Java version: 17.0.12, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-17
Default locale: en_IN, platform encoding: Cp1252
OS name: "windows 10", version: "10.0", arch: "amd64", family: "windows"
  
```

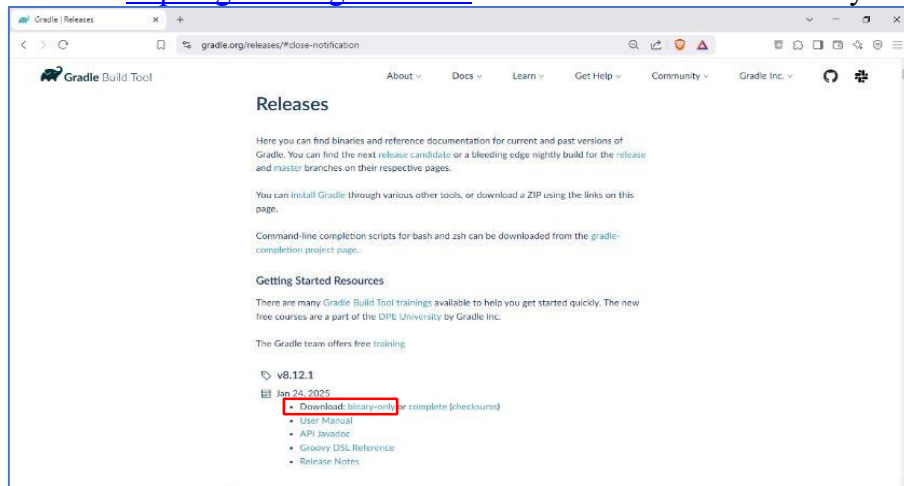

Gradle Installation

Prerequisites

Gradle runs on all major operating systems and requires only a Java JDK version 8 or higher to be installed.

1. Download Gradle:

Visit the <https://gradle.org/releases/> and download the latest binary ZIP file.

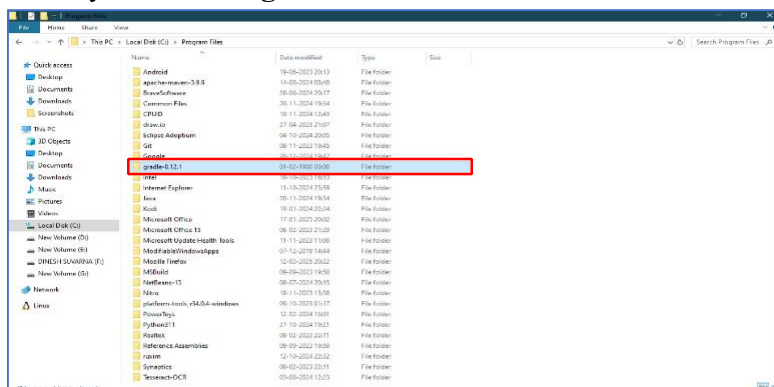


2. Extract the ZIP File:

- Right-click the downloaded ZIP file and select **Extract All**.

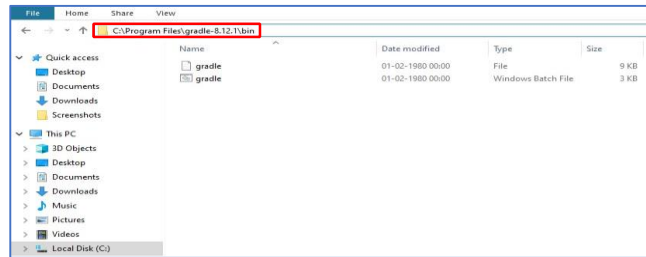


3. **Move the Folder:** o After extraction, move the extracted **Gradle folder (gradle-x.x.x)** to a convenient directory like **C:\Program Files**.



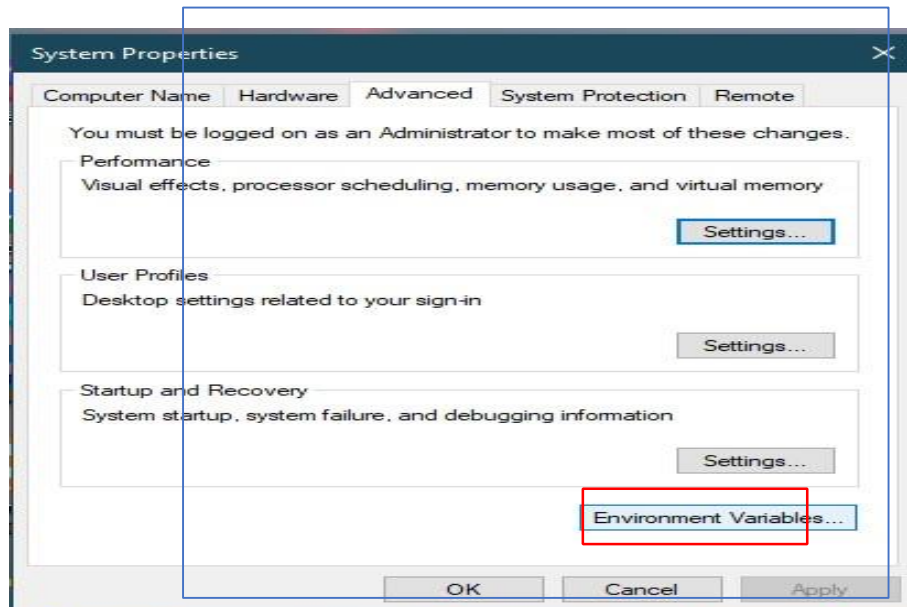
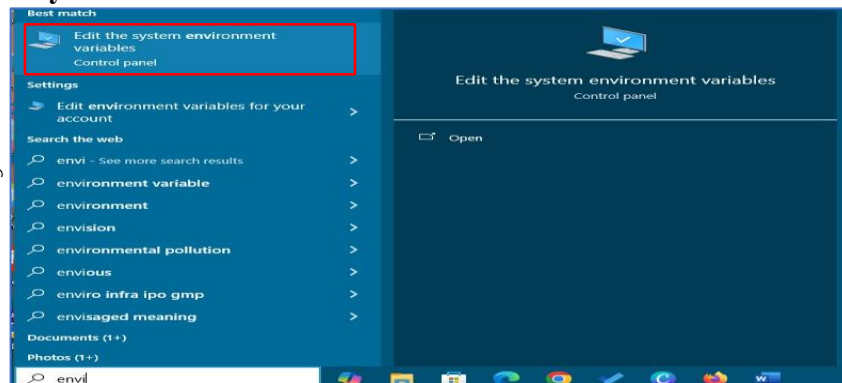
4. Navigate to the bin Folder:

- o Open the **Gradle folder**, then navigate to the **bin** folder inside. o Copy the path from the File Explorer address bar (**C:\Program Files\gradle-x.x\bin**).

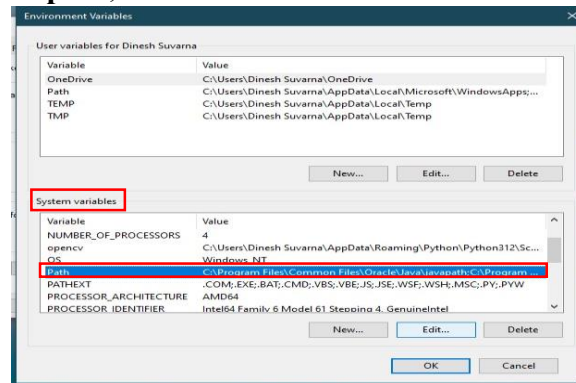


5. Set Environment Variables:

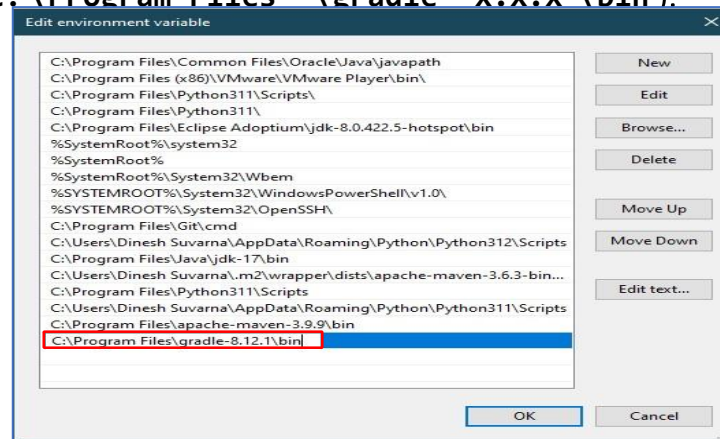
- o Open the **Start Menu**, search for **Environment Variables**, and select **Edit the system environment variables**.



- ✦ Find the **path**, double click on it and click **New**.



- ✦ Paste the full path to the bin folder of your Gradle directory (**C:\Program Files \gradle -x.x.x \bin**).



- o Click **OK** to close the windows and save your changes.

6. Verify the Installation:

1. Open a terminal or Command Prompt and run: **gradle -v** If it shows the Gradle version, the setup is complete.

```

Microsoft Windows [Version 10.0.19045.5487]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Dinesh Suvarna>gradle -v

Welcome to Gradle 8.12.1!

Here are the highlights of this release:
- Enhanced error and warning reporting with the Problems API
- File-system watching support on Alpine Linux
- Build and test Swift 6 libraries and apps

For more details see https://docs.gradle.org/8.12.1/release-notes.html

-----
Gradle 8.12.1
-----
Build time: 2025-01-24 12:55:12 UTC
Revision: 0b1ee1ff81d1f4a26574ff4a362ac9188852b140

Kotlin: 2.0.21
Groovy: 3.0.22
Ant: Apache Ant(TM) version 1.10.15 compiled on August 25 2024
Launcher JVM: 17.0.12 (Oracle Corporation 17.0.12+8-LTS-286)
Daemon JVM: C:\Program Files\Java\jdk-17 (no JDK specified, using current Java home)
OS: Windows 10 10.0 amd64

C:\Users\Dinesh Suvarna>

```

Program 2: Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins

Project Object Model

Pom.xml stands for **Project Object Model**. It's the core configuration file in a **Maven** project. It defines:

- Project metadata (name, version, etc.)
- Dependencies
- Plugins
- Build settings

Explanation of Key Elements

- Mode Version: Defines the POM model version.
- GroupId: A unique identifier for the project's group (e.g., a company or organization).
- ArtifactId: The name of the project, which is used in dependency management.
- Version: Defines the project version (e.g., 1.0.0).
- Packaging: Defines the packaging type (jar, war, pom, etc.).
- Name & description: Optional fields for project metadata.
- Dependencies: Lists external libraries required for the project.
- Build: Contains plugins like maven-compiler-plugin to configure Java versions

Dependency Management and Plugins

Dependency refers to an external library or component that a project relies on in order to function correctly. These external resources provide additional functionality that the project doesn't implement itself but needs in order to work as intended.

For example,

if you're building a Java application and need to parse JSON data, you might include a library like Jackson or Gson. These libraries are dependencies because your project.

A **plugin** in Maven performs **specific tasks** during the build process like **compiling code, running tests, generating documentation, creating JARs, or deploying your app**.

For example,

- Maven-compiler-plugin- Used to compile your Java source files.
- Maven-jar-plugin- Creates JAR files from your compiled code.
- Maven-dependency-plugin- Manages dependencies in your project, such as copying, unpacking, or analyzing them.
- Maven-clean-plugin- Cleans the project by removing previously generated files (like compiled classes, JAR files, etc.).

Procedural Steps:

1. Install Eclipse IDE for Enterprise Java and Web Developer
2. Create a new folder in any drive -> **Ex: D:\FirstProgram**
3. Create a Spring Boot Project in Spring Initializr
4. Download and extract file to same new folder (FirstProgram)
5. Import Spring Boot Project from extracted file in Eclipse IDE
6. Search “maven” and choose Existing Maven Project
7. Browse for the extracted folder and click next
8. Click on the Finish button and we are done creating the Spring Boot project.

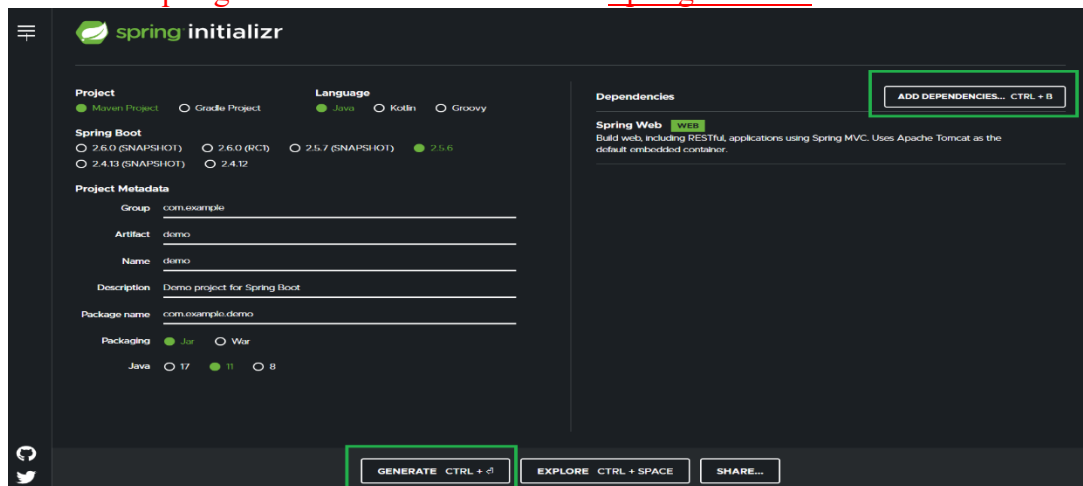
Detailed Explanation:

Step 1: Install Eclipse IDE and create a new folder in any drive **Ex: D:\FirstProgram**

Step 2: Create a Spring Boot Project in Spring Initializr

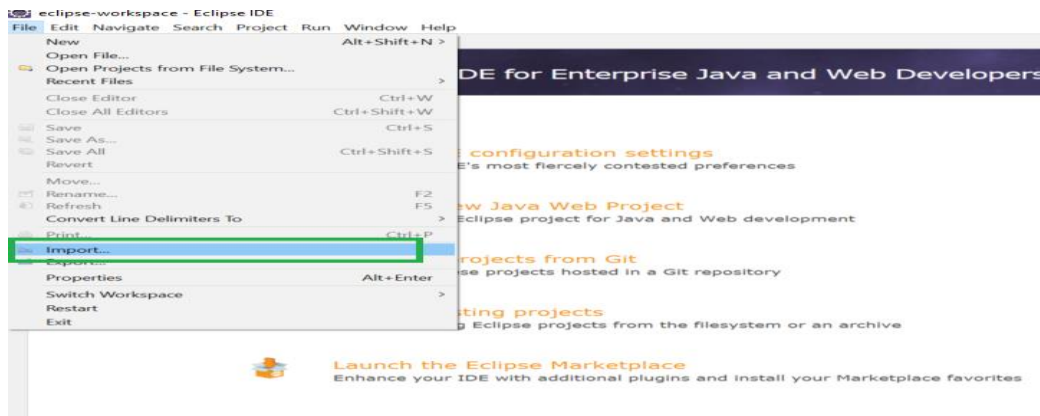
Go to <https://start.spring.io/> and create a Spring Boot project. Please fill in all the details accordingly and at last click on the **GENERATE** button below. This will download your Spring Boot project in zip format. Now extract the folder into your local machine.

For more details in Spring Initializr refer to this article: [Spring Initializr](#)

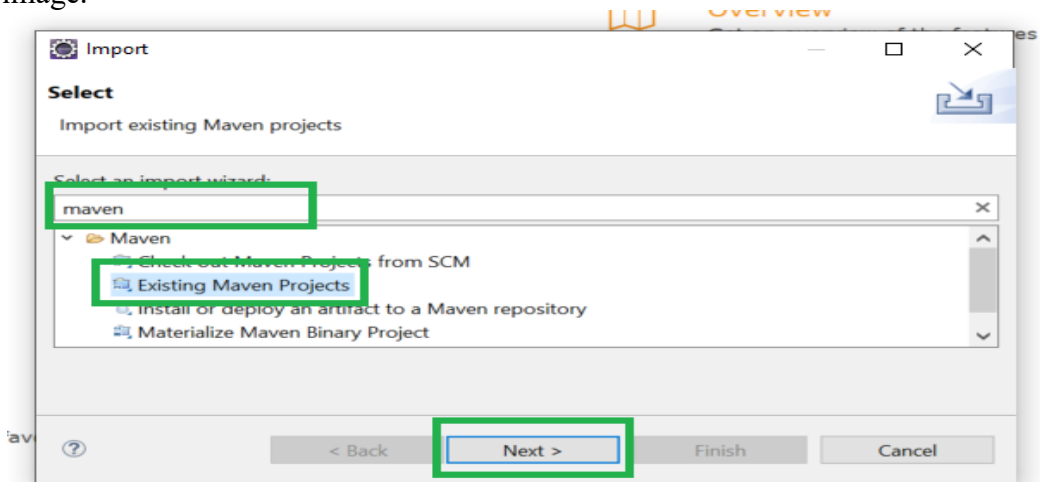


Step 3: Import Spring Boot Project in Eclipse IDE

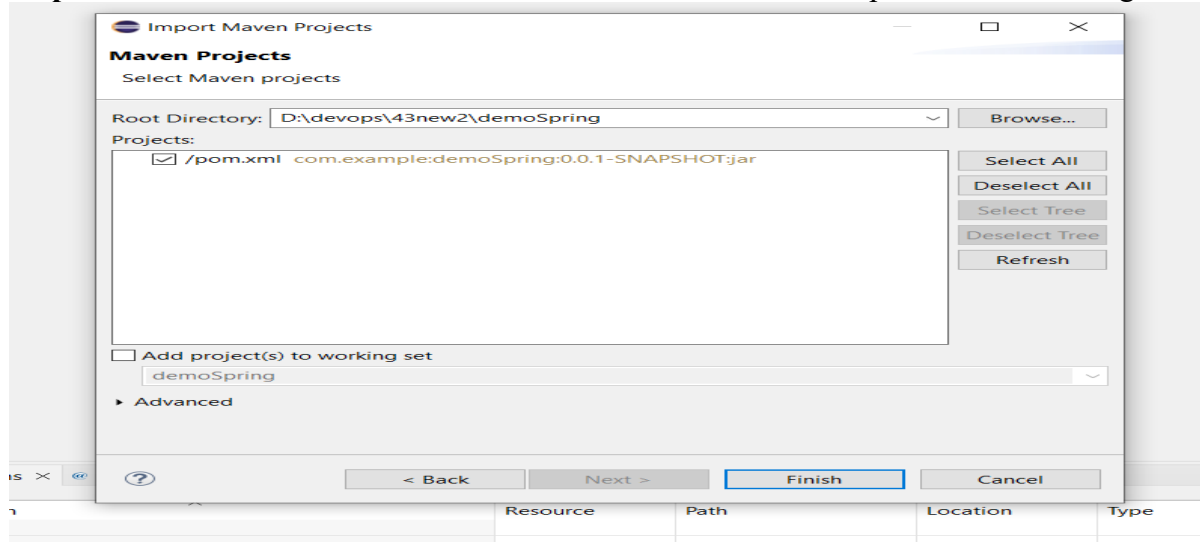
Go to the Eclipse IDE for Enterprise Java and Web Developer > File > Import as shown in the below image.



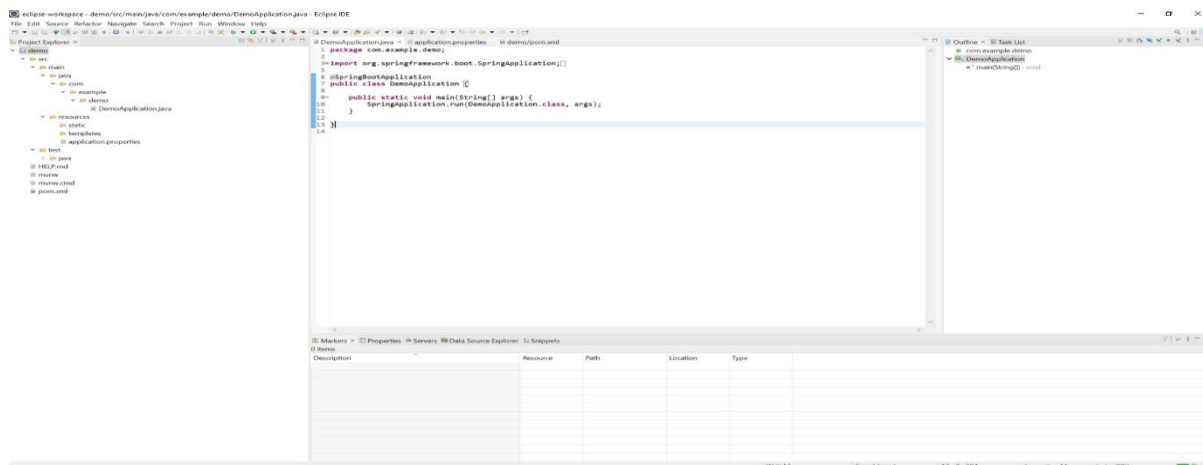
Step 4: Search “maven” and choose Existing Maven Project and click on the **Next** button as shown in the below image.



Step 5: Now click on the **Browse** button and select the extracted zip file that has been generated.



Step 6: Click on the **Finish** button and we are done creating the Spring Boot project.



POM.XML

Step 7: Open pom.xml and add plugin

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <argLine>
      -javaagent:${settings.localRepository}/org/mockito/mockito-
core/${mockito.version}/mockito-core-${mockito.version}.jar
      -Xshare:off
    </argLine>
  </configuration>
</plugin>
```

Step 8: Create a new java class **Ex:HelloController**

package com.example.myfirstspringbootapp;

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
public class HelloController {

    @GetMapping("/")
    public String sayHello() {
return "Hello, World!";
    }
}
```


Step 9: Save the file and refresh

Step 10: Creating jar file

Place the cursor on project -> right click ->run as maven build 1.Clean 2. Install and click refresh

Step 11: Place the cursor on project and right click->terminal

Step 12: Run -> java -jar target\myjar.jar

Step 13: Goto google search and type <http://localhost:8080/>

Note : If error is encountered during application run then click maven build, clean and goto file->restart

Declaring Dependencies

- Dependencies are frameworks required to construct your project.

```
• <dependency>
•   <groupId>org.springframework.boot</groupId>
•   <artifactId>spring-boot-starter-web</artifactId>
• </dependency>
•
• <dependency>
•   <groupId>org.springframework.boot</groupId>
•   <artifactId>spring-boot-starter-test</artifactId>
•   <scope>test</scope>
• </dependency>
```

Program 3: Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation.

Introduction to Gradle

Gradle is a modern build automation tool designed to be highly flexible, fast, and scalable. It is widely used in Java projects, Android development, and many multi-language projects. Here's what makes Gradle stand out:

- **Flexible Build Scripts:** Gradle uses a Domain Specific Language (DSL) based on either Groovy (by default) or Kotlin. This provides a more dynamic and expressive way to define build logic compared to static XML configurations (as used in Maven).
- **Incremental Builds:** Gradle optimizes build times by determining what parts of the project have changed and rebuilding only those parts.
- **Task Automation:** Everything in Gradle is treated as a task, allowing you to create custom tasks or reuse existing ones for compiling code, running tests, packaging, and more.
- **Dependency Management:** Like Maven, Gradle can automatically download and manage dependencies from remote repositories (e.g., Maven Central, JCenter, or custom repositories).

Explanation of Components

- **build.gradle:**
This is the main build script written in Groovy (or Kotlin if you choose). It defines plugins, repositories, dependencies, and tasks.
- **settings.gradle:**
A small script that defines the project's name and, in multi-project builds, the included subprojects.
- **gradlew / gradlew.bat:**
The Gradle wrapper scripts. They allow you to run Gradle without requiring a separate installation on every machine by automatically downloading the correct Gradle version.
- **src/main/java:**
Contains your application's source code.
- **src/test/java:**
Contains your unit tests.

Understanding Gradle Build Scripts

A **build script** in Gradle is a programmatic file that instructs Gradle on how to build your project. It can be written in two main DSLs:

- **Groovy DSL (build.gradle):** The traditional and most common syntax.
- **Kotlin DSL (build.gradle.kts):** A statically-typed alternative that leverages Kotlin's language features.

Dependency Management in Gradle How It Works

Repositories: Gradle downloads dependencies from defined repositories (e.g., Maven Central).

Dependency Notation: Dependencies are defined in a simple format:

group:artifact:version.

Configuration Types: Gradle provides various configurations such as `implementation`, `compileOnly`, `runtimeOnly`, and `testImplementation` to manage the scope of each dependency.

Example:

In our build script examples, we included JUnit for testing:

```
dependencies {  
    testImplementation 'junit:junit:4.13.2'  
}
```

This line tells Gradle to download JUnit version 4.13.2 from Maven Central and include it in the test classpath.

Task Automation in Gradle

In Gradle, nearly everything is a task. Tasks represent individual units of work (compiling code, running tests, packaging applications). Gradle comes with many built-in tasks (provided by plugins) and allows you to define your own custom tasks.

Built-in Tasks:

`compileJava`: Compiles the source code in `src/main/java`.

`test`: Runs tests in `src/test/java`.

`Jar`: Packages compiled code into a JAR file.

`Run`: Runs the application (if the application plugin is applied).

Creating Custom Tasks:

Custom Task in Groovy DSL:

Ensure your build.gradle includes the following custom task:

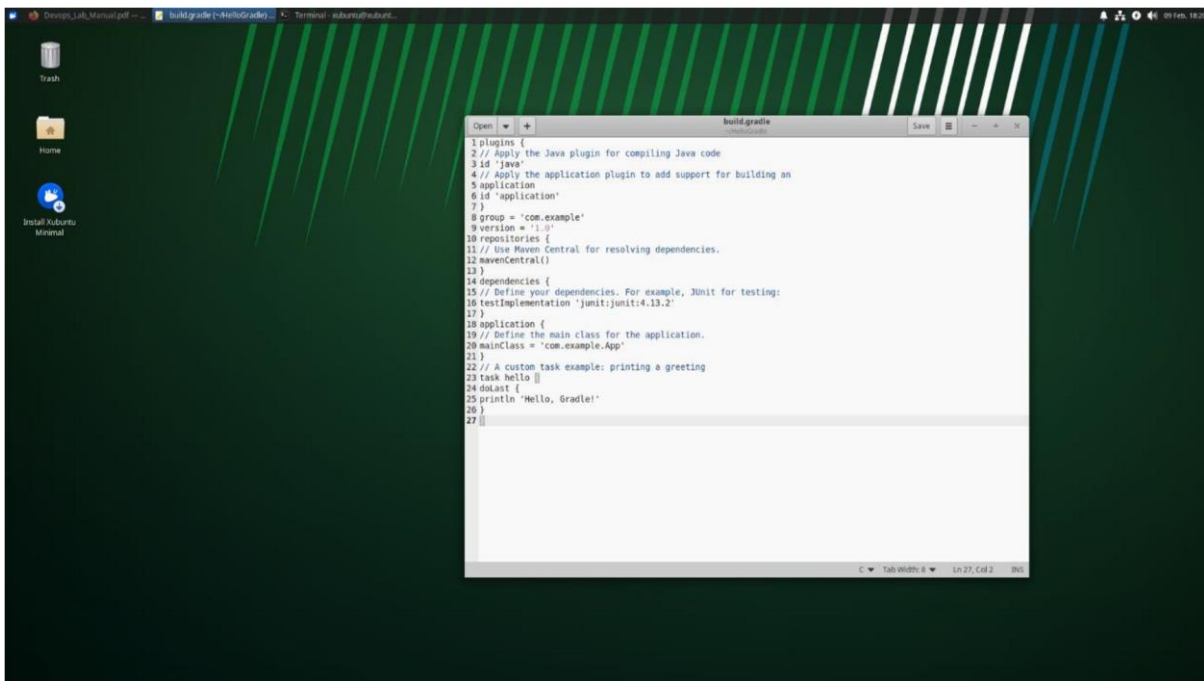
```
task hello {  
    doLast {  
        println 'Hello, Gradle!'  
    }  
}
```

Run the Custom Task:

Command: **gradle hello**

Expected Output:

Hello, Gradle!



STEP1:

Goto Command Prompt

Then first make a new directory the command is

```
mkdir pgm3
```

For changing to a current directory the command is

```
cd pgm3
```

Now run

```
gradle init
```

After execution of command the screen shows as in below where we opt for build type select as 1

```
C:\Users\CMRIT-ISE-L209-009\gradletest>gradle init
Starting a Gradle Daemon (subsequent builds will be faster)

Select type of build to generate:
 1: Application
 2: Library
 3: Gradle plugin
 4: Basic (build structure only)
Enter selection (default: Application) [1..4] 1
```

After selecting application type next it asks for Implementation language select as groovy

```
Select implementation language:
 1: Java
 2: Kotlin
 3: Groovy
 4: Scala
 5: C++
 6: Swift
Enter selection (default: Java) [1..6] 3
```

After selecting Implementation language it will ask for Java version and project name

```
Enter selection (default: Java) [1..6] 3

Enter target Java version (min: 7, default: 21): 21

Project name (default: gradletest): gradletest
```

After providing version and project name

Select application structure as Single application structure and Domain

Specific Language as Kotlin

```
Select application structure:
  1: Single application project
  2: Application and library project
Enter selection (default: Single application project) [1..2] 1

Select build script DSL:
  1: Kotlin
  2: Groovy
Enter selection (default: Groovy) [1..2] 1
```

After every procedure is over it shows Build successful

```
Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no] yes

> Task :init
Learn more about Gradle by exploring our Samples at https://docs.gradle.org/8.12.1/samples/sample\_building\_groovy\_applications.html

BUILD SUCCESSFUL in 2m 2s
1 actionable task: 1 executed
```

STEP2: Now it's time to Build the script

Just type the command as:

gradlew run

It will take atleast 3-5 minutes to run the configuration script the output be as in below.

If You want to see the structure of an application run the command as

tree

```
C:\Users\CMRIT-ISE-L209-009\gradletest>gradlew run
Calculating task graph as no cached configuration is available for tasks: run

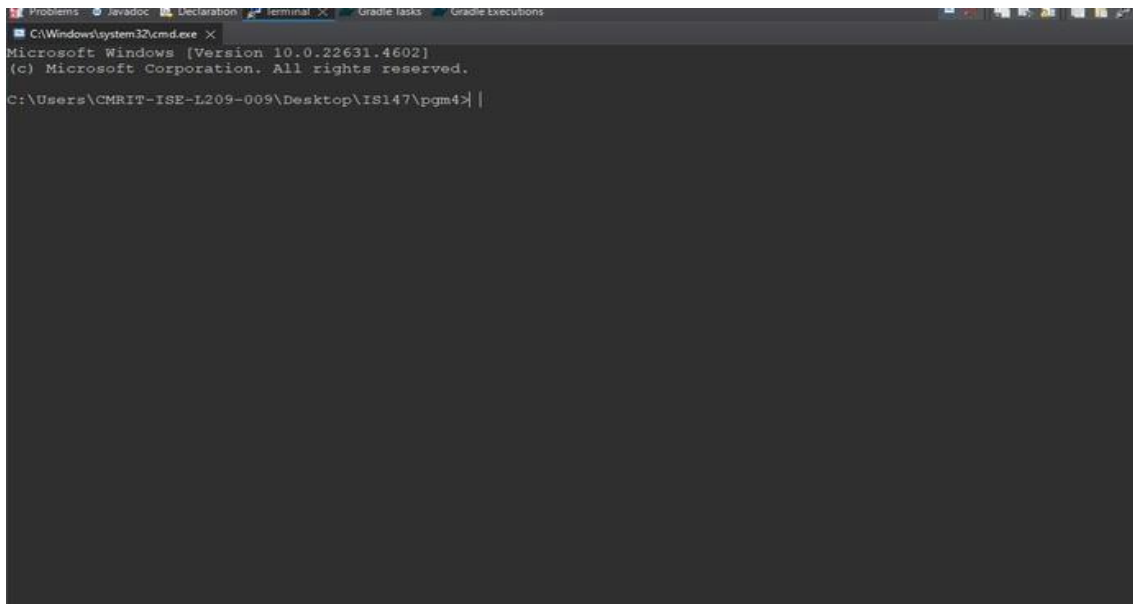
> Task :app:run
Hello World!
```

```
C:\Users\CMRIT-ISE-L209-009\gradletest>tree
Folder PATH listing for volume Windows
Volume serial number is CA13-EB08
C:.
|-- .gradle
|   |-- 8.12.1
|   |   |-- checksums
|   |   |-- executionHistory
|   |   |-- expanded
|   |   |-- fileChanges
|   |   |-- fileHashes
|   |   |-- vcsMetadata
|   |-- buildOutputCleanup
|   |-- configuration-cache
|   |   |-- 87330068-729b-48ed-8a38-57771bbaae67
|   |   |-- 8qybe7c3ykh3sf9t2slk1e4w
|   |-- vcs-1
|-- .settings
|-- app
|   |-- build
|   |   |-- classes
|   |   |   |-- groovy
|   |   |   |   |-- main
|   |   |   |   |   |-- org
|   |   |   |   |   |   |-- example
|   |   |-- generated
|   |   |   |-- sources
|   |   |   |   |-- annotationProcessor
|   |   |   |   |-- groovy
|   |   |   |   |   |-- main
|   |   |-- tmp
|   |   |   |-- compileGroovy
|   |   |   |   |-- groovy-java-stubs
|-- src
|   |-- main
|   |   |-- groovy
|   |   |   |-- org
|   |   |   |   |-- example
|   |   |-- resources
|   |-- test
|   |   |-- groovy
|   |   |   |-- org
```

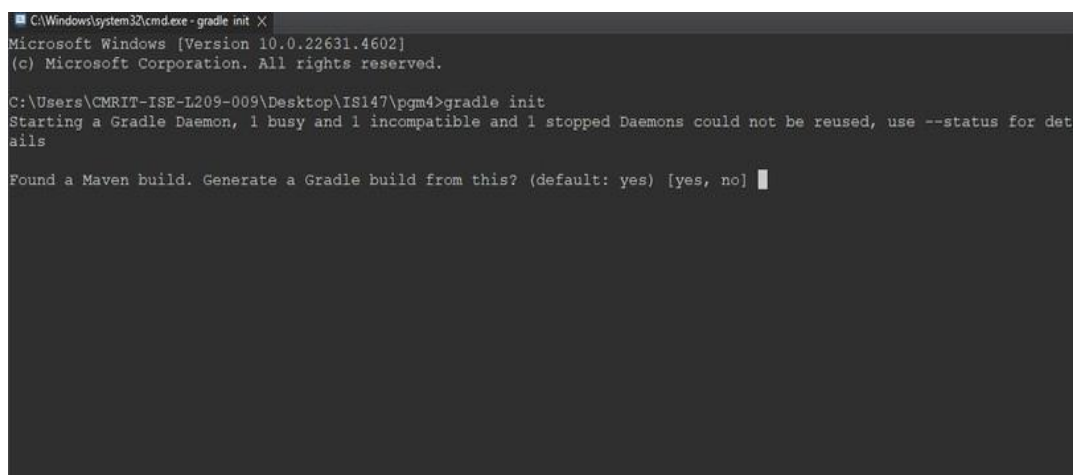

Program 4: Practical Exercise: Build and Run a Java Application with Maven, Migrate the Same Application to Gradle.

STEP1: First create a Maven Project as in PROGRAM2 then build the project and run java application you will get Hello World Message

STEP2: Then to migrate to gradle use shortcut Key Ctrl+Alt+Shif+T To get Terminal screen as in below:



STEP3: Type command **gradle init** it will ask for migrate from maven to gradle type yes



STEP4: After the above command is validated to yes it prompts to **select Domain Specific Language** as in screen below select 2 (as we have done for Kotlin)

```
Select build script DSL:
 1: Kotlin
 2: Groovy
Enter selection (default: Kotlin) [1..2]
```

STEP5:After selecting Groovy it asks for validating prompt for API Generator just validate as yes

```
C:\Windows\system32\cmd.exe - gradle init X
C:\Users\CMRIT-ISE-L209-009\Desktop\IS147\pgm4>gradle init

Found a Maven build. Generate a Gradle build from this? (default: yes) [yes, no] yes

Select build script DSL:
 1: Kotlin
 2: Groovy
Enter selection (default: Kotlin) [1..2] 2

Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no] ||
```

Finally it runs the init phase as been selected

STEP 6: Type the command gradle build

```
C:\Windows\system32\cmd.exe X
C:\Users\CMRIT-ISE-L209-009\Desktop\IS147\pgm4>gradle build
Reusing configuration cache.

BUILD SUCCESSFUL in 759ms
3 actionable tasks: 4 up-to-date
Configuration cache entry reused.
C:\Users\CMRIT-ISE-L209-009\Desktop\IS147\pgm4>
```

Now to get exact program output of our java file Locate to build gradle File from ur local repository and Copy paste the code as in below shown in red color

```
plugins {
    id 'java-library'
    id 'maven-publish'
    id 'application'
}
```

```
application {
    mainClass = 'com.ppg4.ppg4.App' // Specify the main class here
}
```

```
repositories {
    mavenLocal()
    maven {
        url = uri('https://repo.maven.apache.org/maven2/')
    }
}
```

```

    }
}

dependencies {
    testImplementation libs.junit.junit
}

group = 'com.ppg4'
version = '0.0.1-SNAPSHOT'
description = 'ppg4'
java.sourceCompatibility = JavaVersion.VERSION_1_8

publishing {
    publications {
        maven(MavenPublication) {
            from(components.java)
        }
    }
}

tasks.withType(JavaCompile) {
    options.encoding = 'UTF-8'
}

tasks.withType(Javadoc) {
    options.encoding = 'UTF-8'
}

```

After doing all changes final step

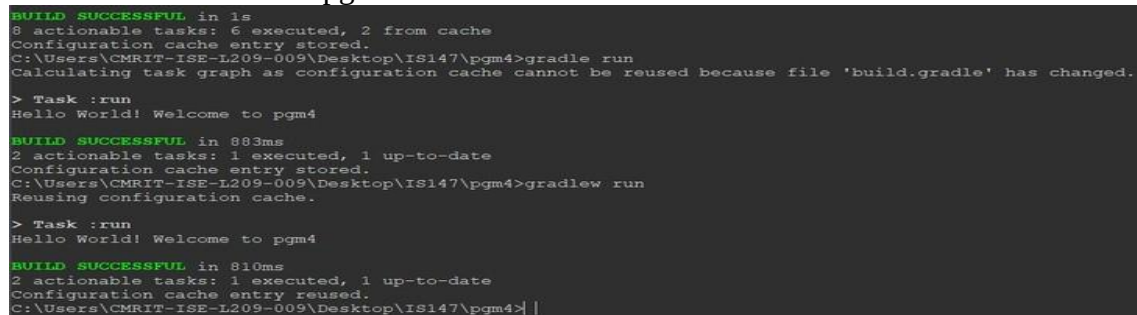
To run commands

gradle clean build

gradle run

You will get Output as

Hello World! Welcome to pgm4



```

BUILD SUCCESSFUL in 1s
8 actionable tasks: 6 executed, 2 from cache
Configuration cache entry stored.
C:\Users\CMRIT-ISE-L209-009\Desktop\IS147\pgm4>gradle run
Calculating task graph as configuration cache cannot be reused because file 'build.gradle' has changed.
> Task :run
Hello World! Welcome to pgm4

BUILD SUCCESSFUL in 883ms
2 actionable tasks: 1 executed, 1 up-to-date
Configuration cache entry stored.
C:\Users\CMRIT-ISE-L209-009\Desktop\IS147\pgm4>gradlew run
Reusing configuration cache.
> Task :run
Hello World! Welcome to pgm4

BUILD SUCCESSFUL in 810ms
2 actionable tasks: 1 executed, 1 up-to-date
Configuration cache entry reused.
C:\Users\CMRIT-ISE-L209-009\Desktop\IS147\pgm4> |

```

Program 5: Introduction to Jenkins: What is Jenkins?, Installing Jenkins on Local or Cloud Environment, Configuring Jenkins for First Use.

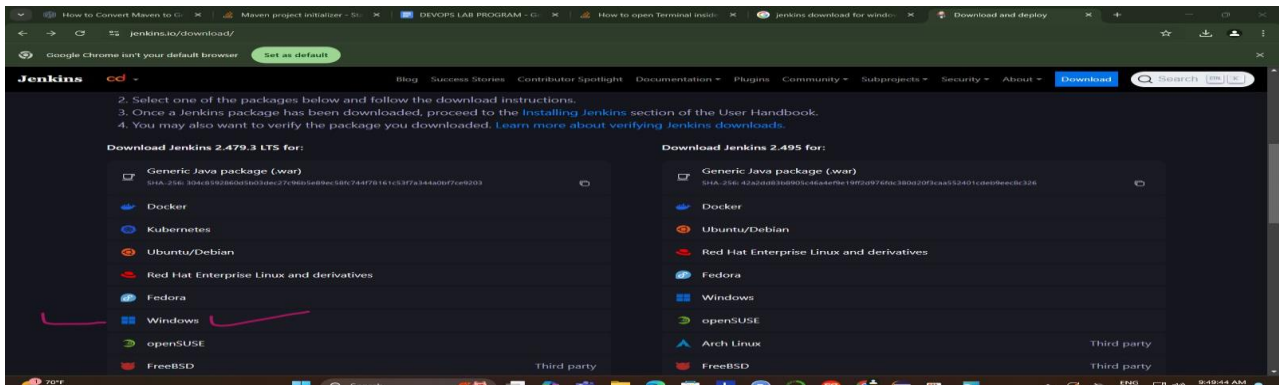
What Is Jenkins?

Jenkins is an open-source automation server written in Java. It is widely used to facilitate Continuous Integration (CI) and Continuous Deployment/Delivery (CD) pipelines in software development. Jenkins automates repetitive tasks related to building, testing, and deploying software, helping teams to integrate changes continuously and deliver high-quality applications faster.

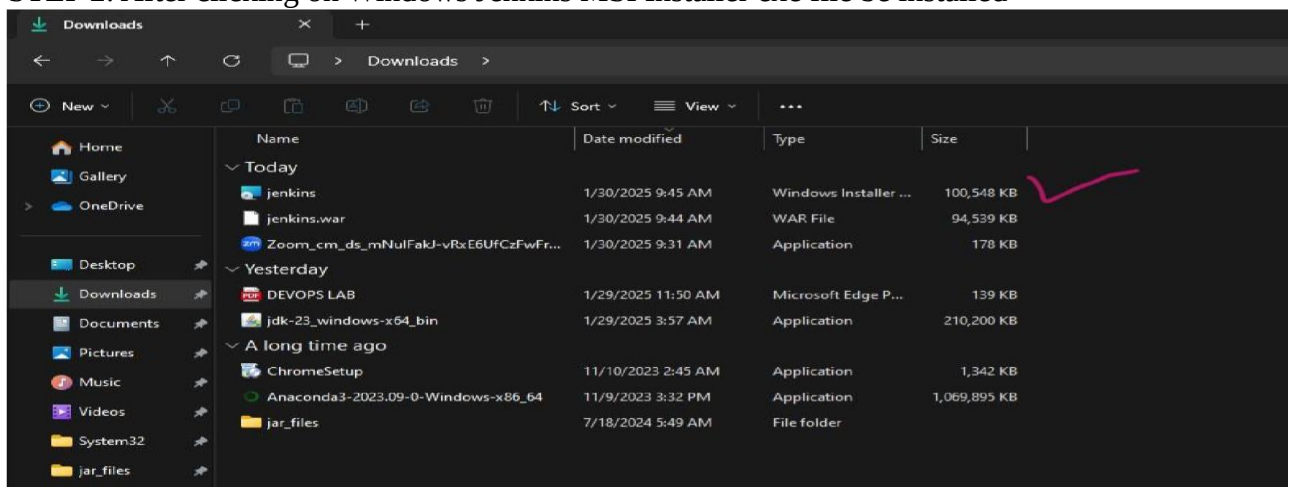
Key Functionalities

- **Continuous Integration/Delivery:** Automatically builds, tests, and deploys code after each commit.
- **Extensible Plugin Ecosystem:** Over 1,500 plugins allow integration with numerous tools such as Git, Maven, Gradle, Docker, and many cloud providers.
- **Automated Builds:** Jenkins can poll version control systems, trigger builds on code changes, and even schedule builds.
- **Pipeline as Code:** With the introduction of Jenkins Pipeline (using either scripted or declarative syntax), you can define your entire build process in a code file (Jenkinsfile) and store it alongside your code.
- **Distributed Builds:** Jenkins can distribute workloads across multiple machines, helping to run tests and builds faster.
- **Monitoring and Reporting:** Provides detailed logs, test reports, and build history.

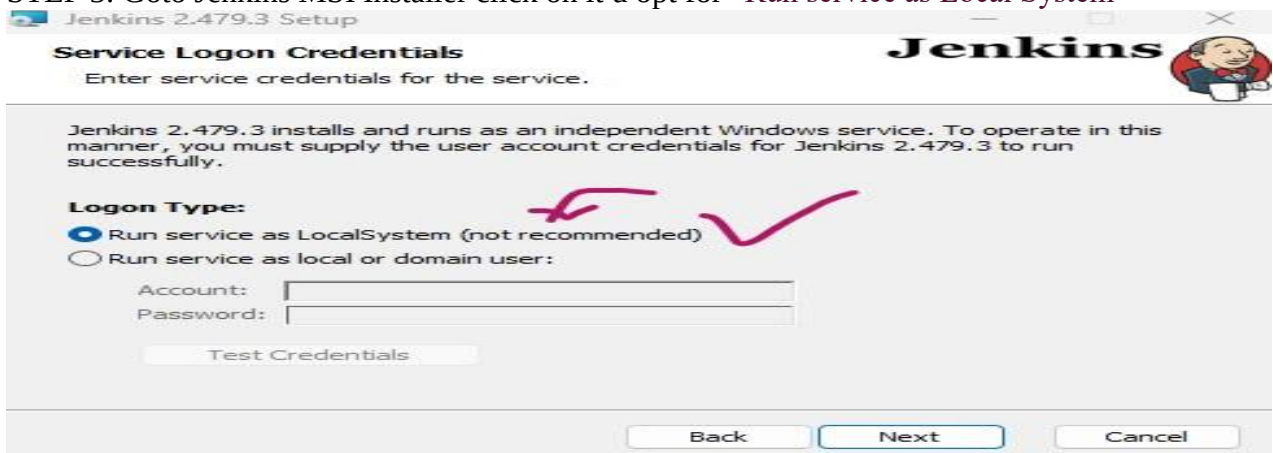
STEP 1: Type jenkins download for windows <https://www.jenkins.io/download/>



STEP 2: After clicking on Windows Jenkins MSI Installer exe file be installed

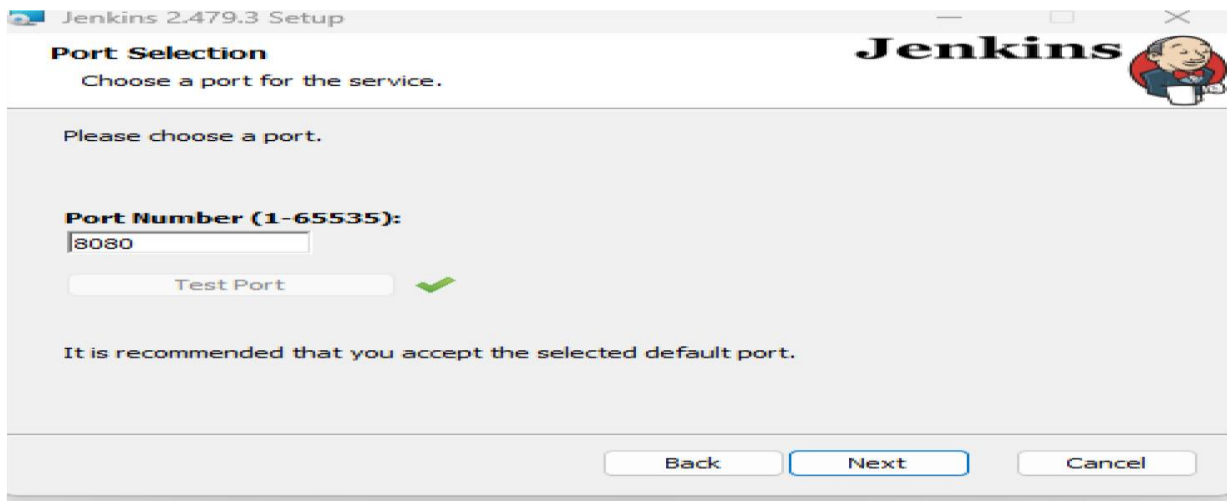


STEP 3: Goto Jenkins MSI Installer click on it u opt for "Run service as Local System"



XX Need not to Provide Account & Password XX

STEP 4: Choose a port as 8080 and test the port and click Next



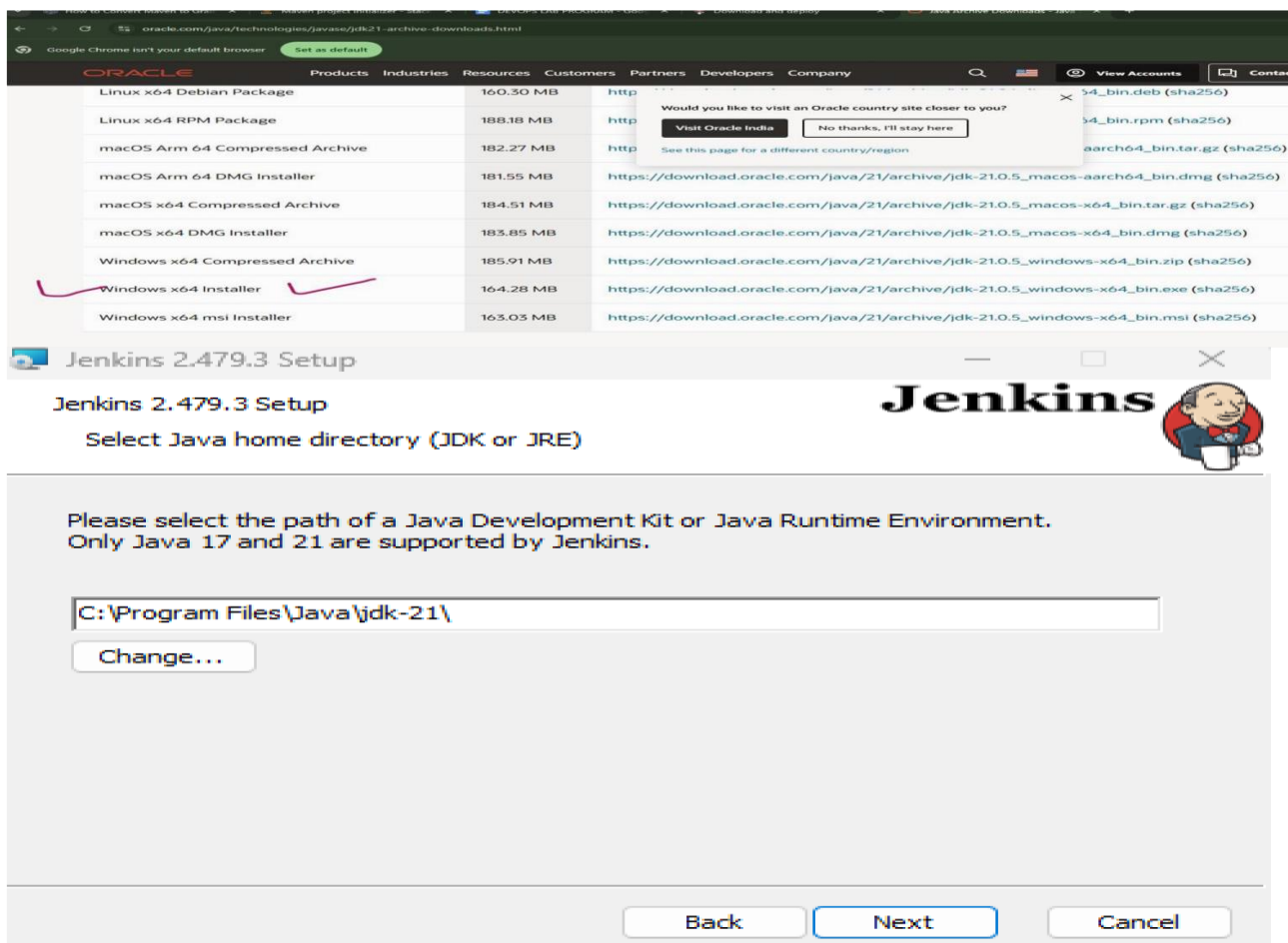
STEP 5: It takes current jdk version that's available for safer side once goto cmd prompt and type command

java -version

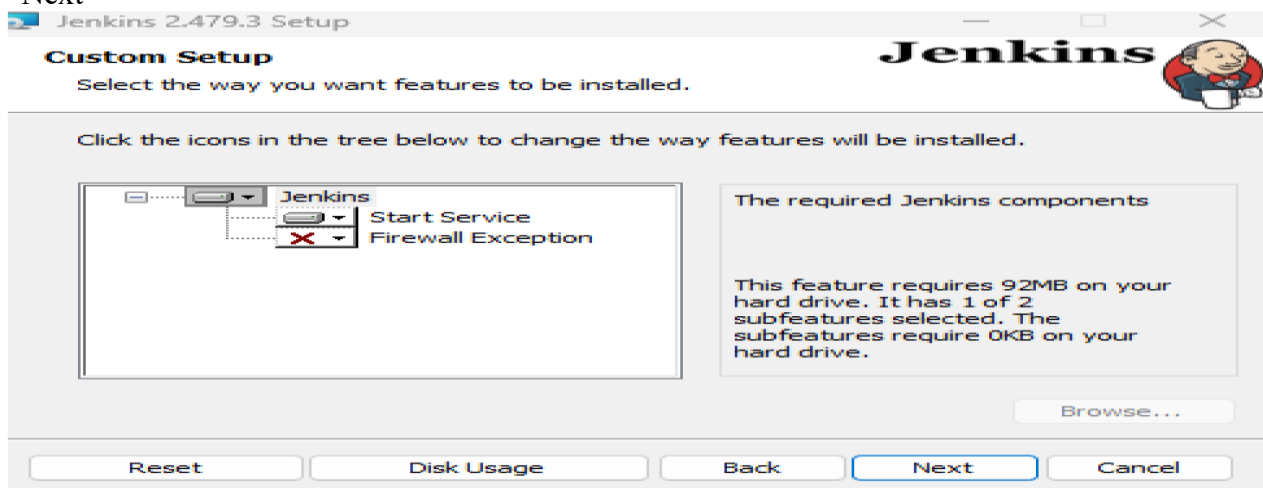
If matching click next

If failed to accept download jdk version 17 to 21 any of it

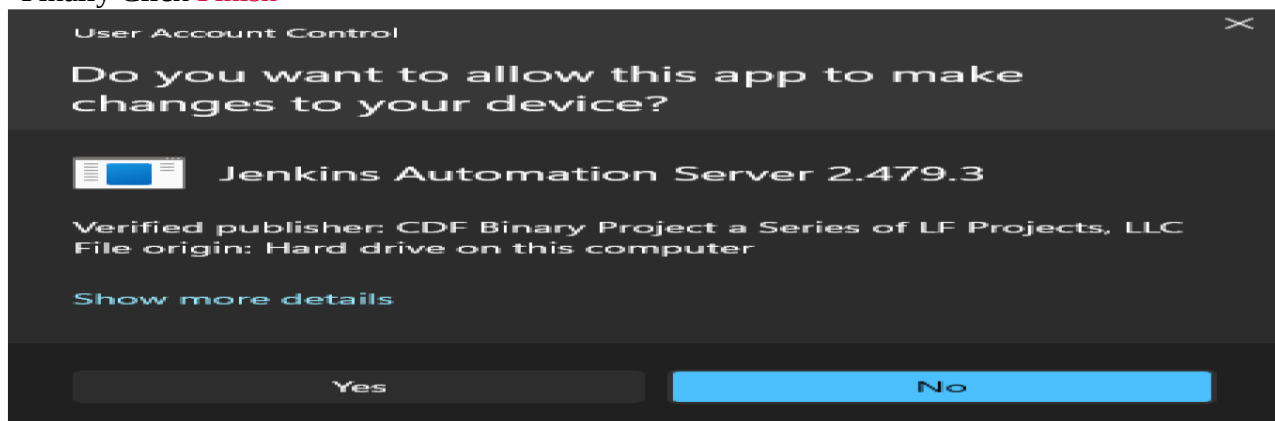
<https://www.oracle.com/java/technologies/javase/jdk21-archive-downloads.html>



STEP 6: Click NEXT after doing above step you will get screen as in Below again continue to click Next



STEP 7: The popup allow format comes for Java Automation Server allow it click **install** ----- > Finally Click **Finish**



Now By default Our Jenkins run at <http://localhost:8080/>

Final step very important Once  It will ask for Administrator Password so u should locate as in directory mention copy paste as in mention in file location:
C:\ProgramData\Jenkins\jenkins\secrets\initialAdminPassword and paste password as in mentioned belows administration password

user name :admin

password: (secret code)

Then select Install Suggested Plugins it starts to install as in shown below

Getting Started

Getting Started

✓ Folders	✓ OWASP Markup Formatter	✓ Build Timeout	✓ Credentials Binding	- OWASP Markup Formatter ** ASM API ** JSON Path API ** Struts ** Pipeline: Step API ** Token Macro
✓ Timestampers	Workspace Cleanup	Ant	Gradle	- Build Timeout ** bouncycastle API ** Credentials ** Plain Credentials ** Variant ** SSH Credentials
Pipeline	GitHub Branch Source	Pipeline: GitHub Groovy Libraries	Pipeline Graph View	- Credentials Binding ** SCM API ** Pipeline: API ** commons-lang3 v3.x Jenkins API
Git	SSH Build Agents	Matrix Authorization Strategy	PAM Authentication	- Timestampers ** Caffeine API ** Script Security ** JavaBeans Activation Framework (JAF) API ** JAXB ** SnakeYAML API ** - required dependency
LDAP	Email Extension	Mailer	Dark Theme	

Jenkins 2.479.3

Getting Started

Create First Admin User

Username

Password

Confirm password

Full name

E-mail address

Jenkins 2.479.3

[Skip and continue as admin](#) [Save and Continue](#)

Final screen be:

Getting Started

Jenkins is ready!

You have skipped the **setup of an admin user**.

To log in, use the username: "admin" and the administrator password you used to access the setup wizard.

Your Jenkins setup is complete.

[Start using Jenkins](#)

Jenkins 2.479.3

Program 6: Continuous Integration with Jenkins: Setting Up a CI Pipeline, Integrating Jenkins with Maven/Gradle, Running Automated Builds and Tests

What is a CI Pipeline

A Continuous Integration (CI) Pipeline automates the process of building, testing, and integrating code changes every time code is committed to the repository. This pipeline:

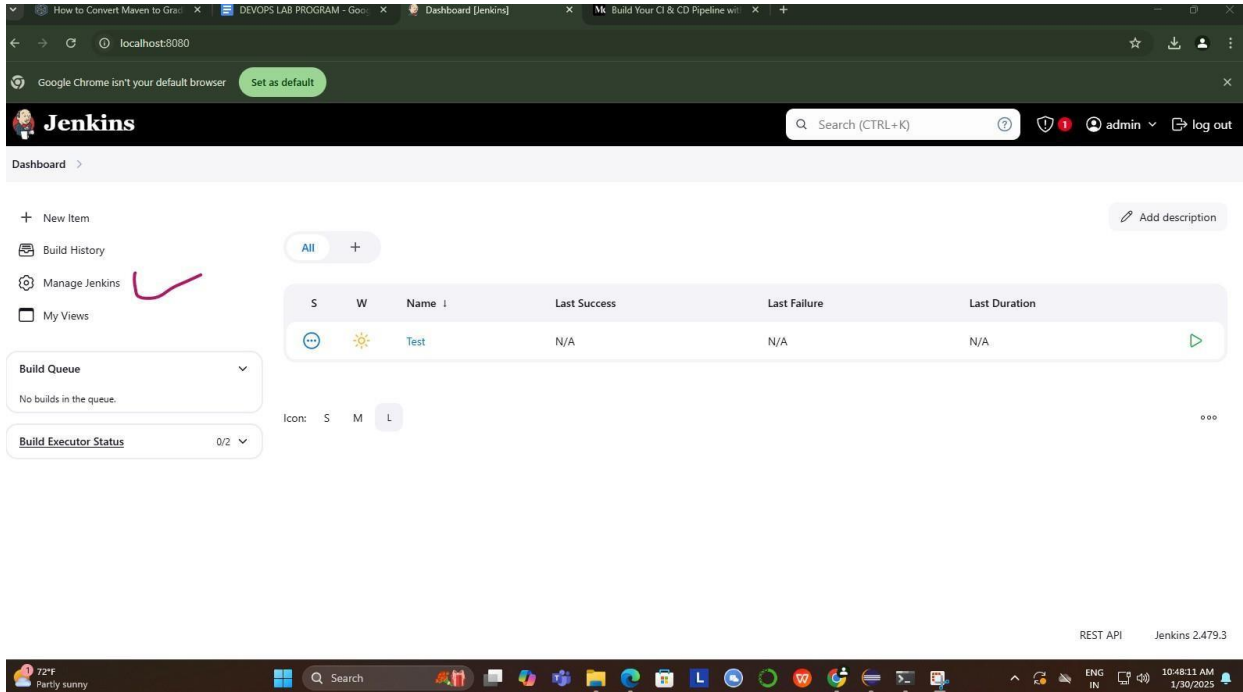
- Automatically checks out the latest code.
- Compiles the application.
- Runs tests to catch errors early.
- Notifies the team of build/test results.

Why Use Jenkins for CI?

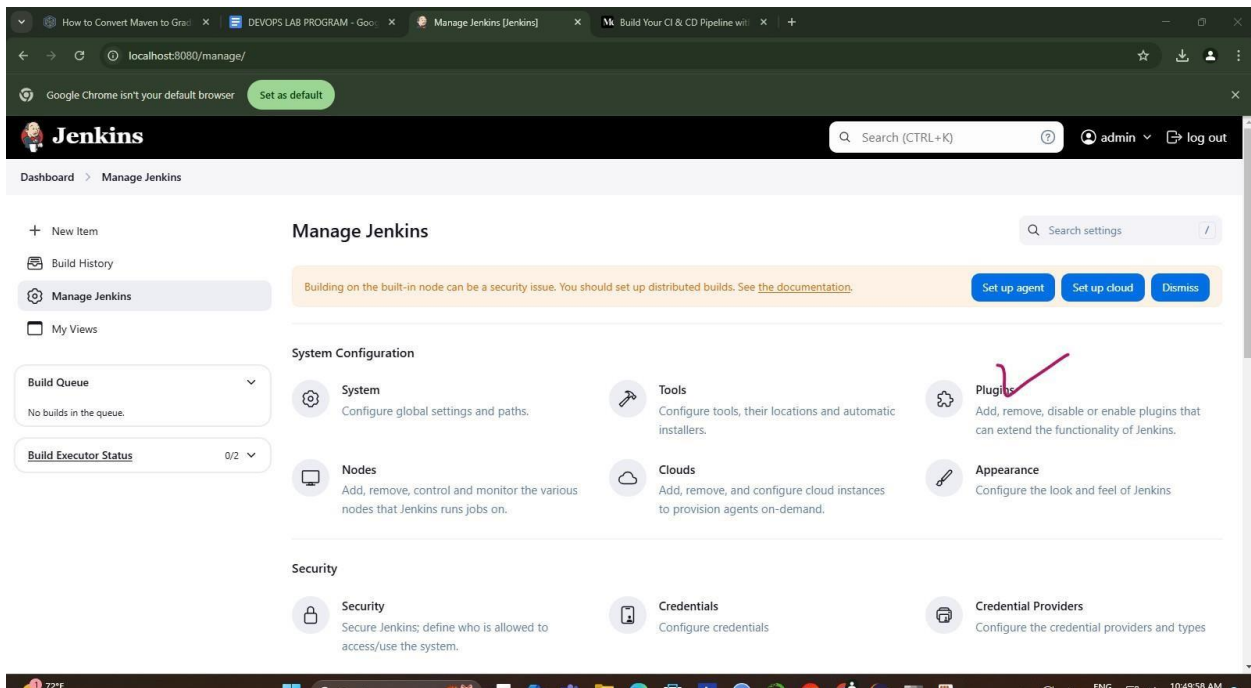
- **Automation:** Jenkins automates the build and test cycle, reducing manual intervention.
- **Immediate Feedback:** Developers get rapid notifications of any integration issues.
- **Extensibility:** With hundreds of plugins available, Jenkins can integrate with version control systems, build tools (Maven, Gradle), testing frameworks, and more.
- **Pipeline as Code:** Using Jenkins Pipelines (defined in a `Jenkinsfile`), you can manage the CI process as part of your source code repository.

Setting Up a CI Pipeline with Jenkins (Freestyle Project)

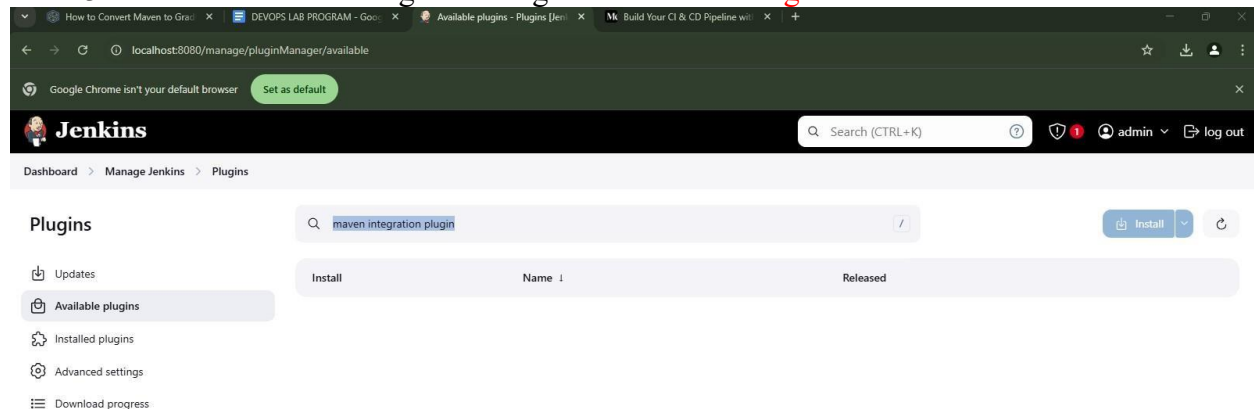
STEP1: Now coming to our Program to set CI Pipeline for Maven Go to Jenkins Dashboard and Click on Manage Jenkins



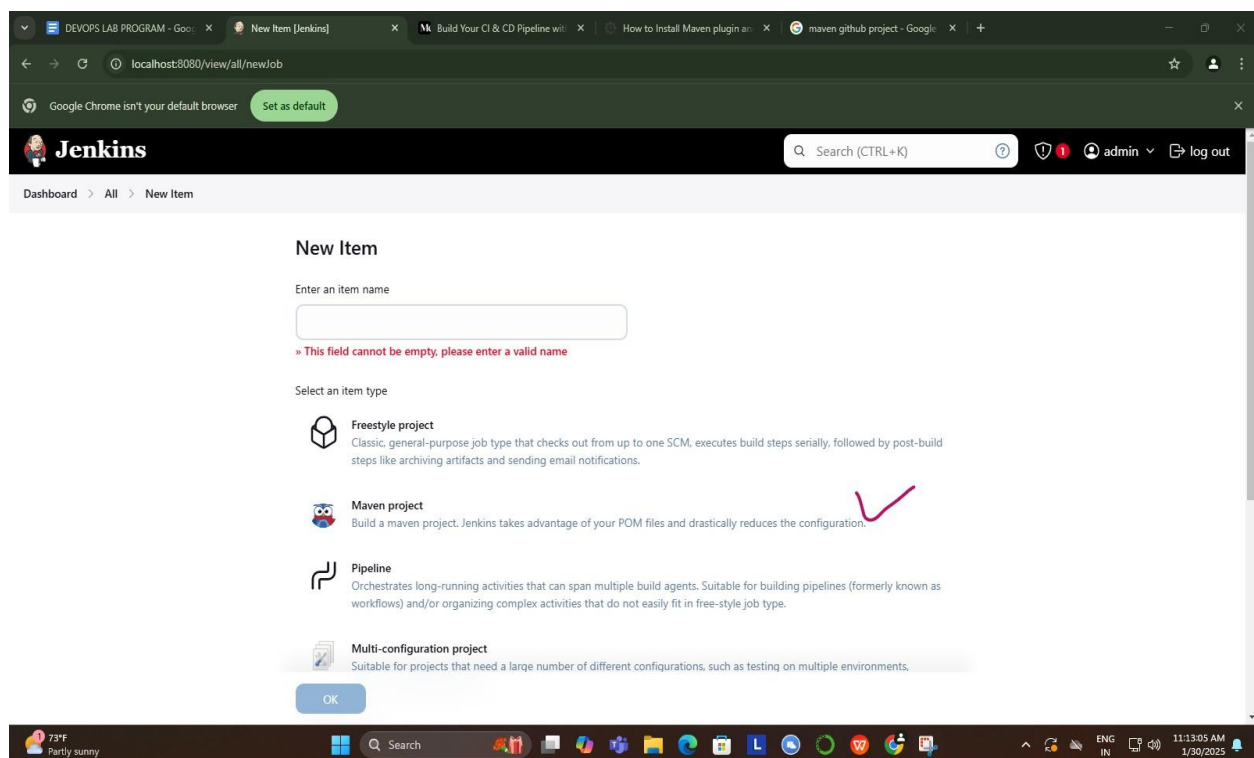
STEP2: Select Plugins



STEP3: Search for Maven IntegrationPlugin in Available Plugins and Install

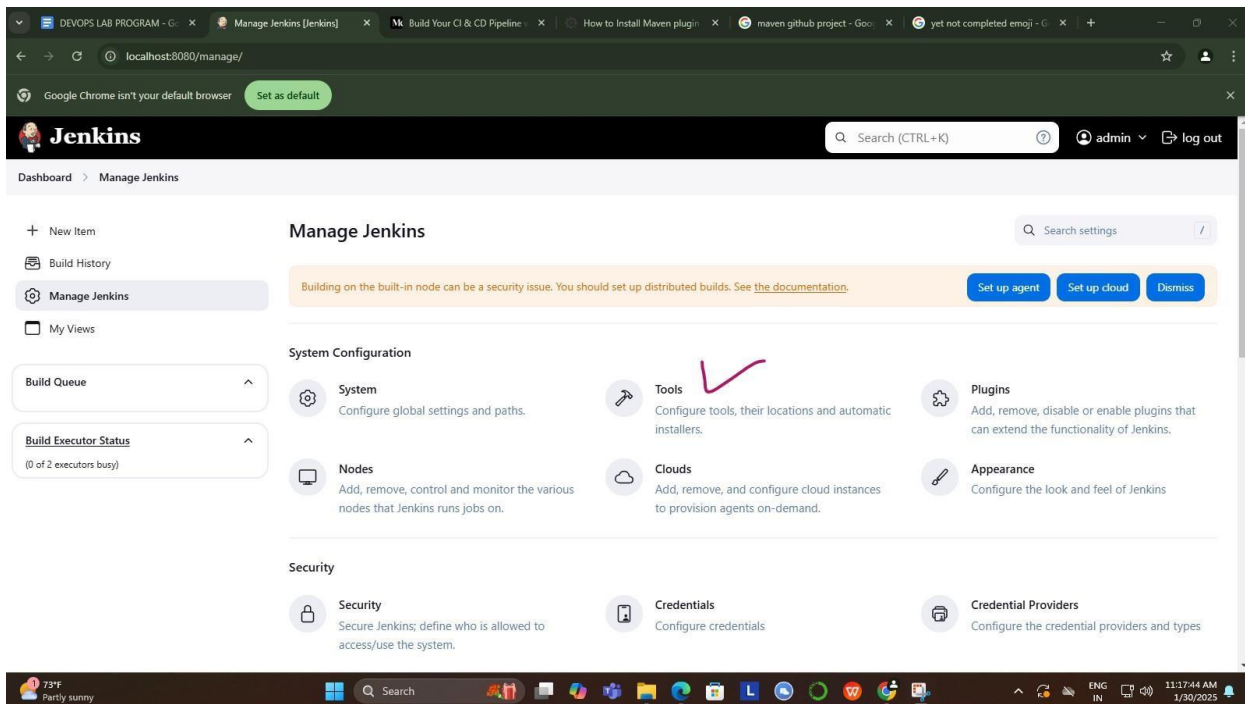


STEP4: After Maven Integration Plugin is Installed We able to see Maven Project as New Item



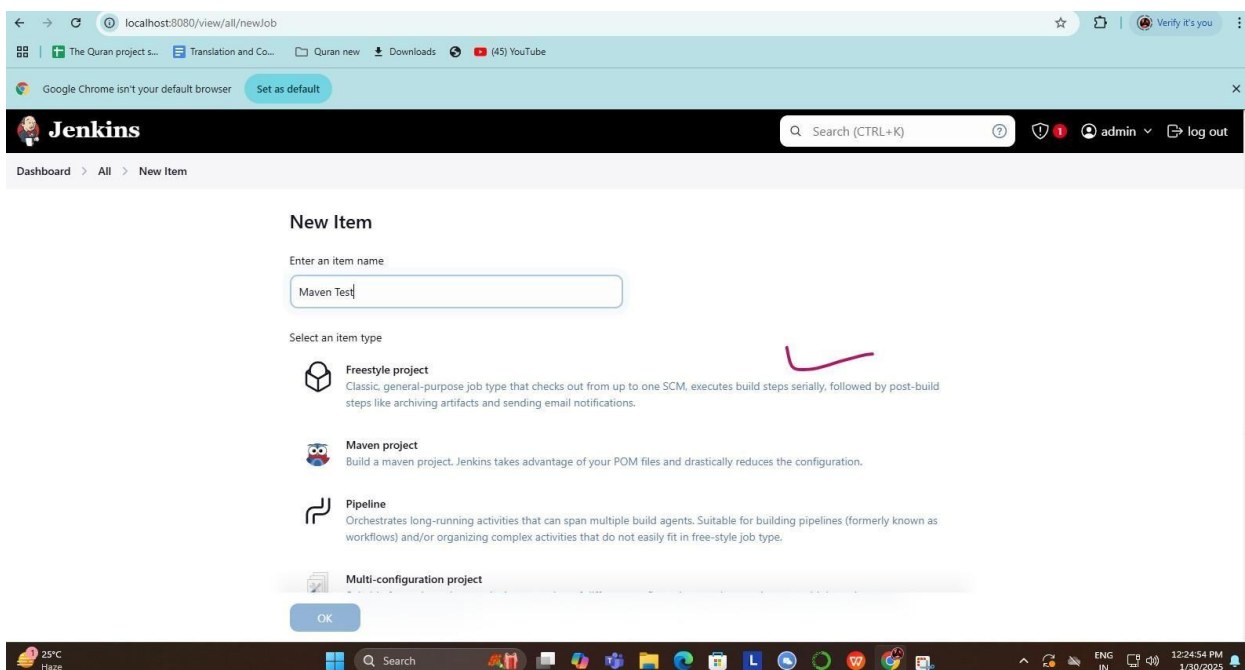


STEP5: YET not completed we have to configure the Location to properly Build and Run Maven Project. So again click on Manage Jenkins and select Tools

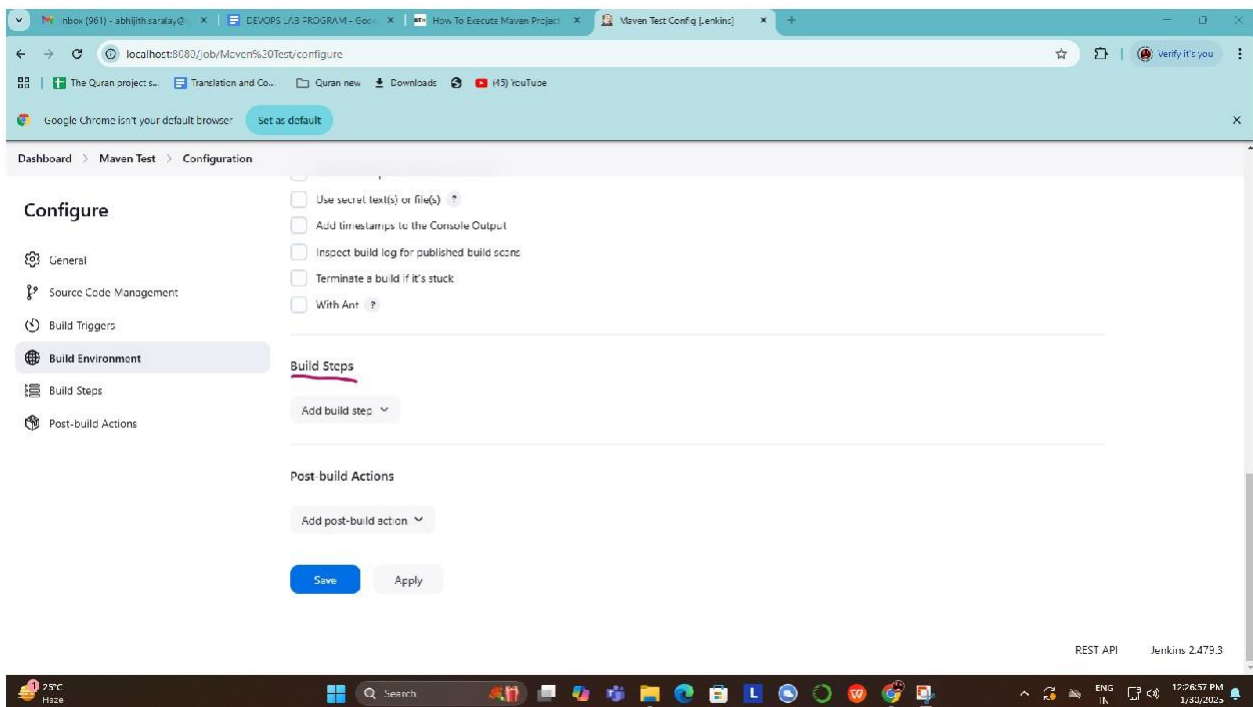


STEP6: Now lets not select Maven Project as new Item as we already have Maven project in local systems lets see how we can run the Maven Project with POM.XML

- Click on New Item
- Provide Item Name and select Freestyle Project



- c. Scroll down to 'Build' option. Click on 'Add Build Step' and choose the value 'Invoke top-level Maven targets' from the drop down list.



After selecting Invoke top-level Maven targets opt for proper environment version as in set in previous steps in my case its MAVEN_HOME

Build Steps

☰

Invoke top-level Maven targets ?

✕

Maven Version

MAVEN_HOME

Goals

Advanced ▾

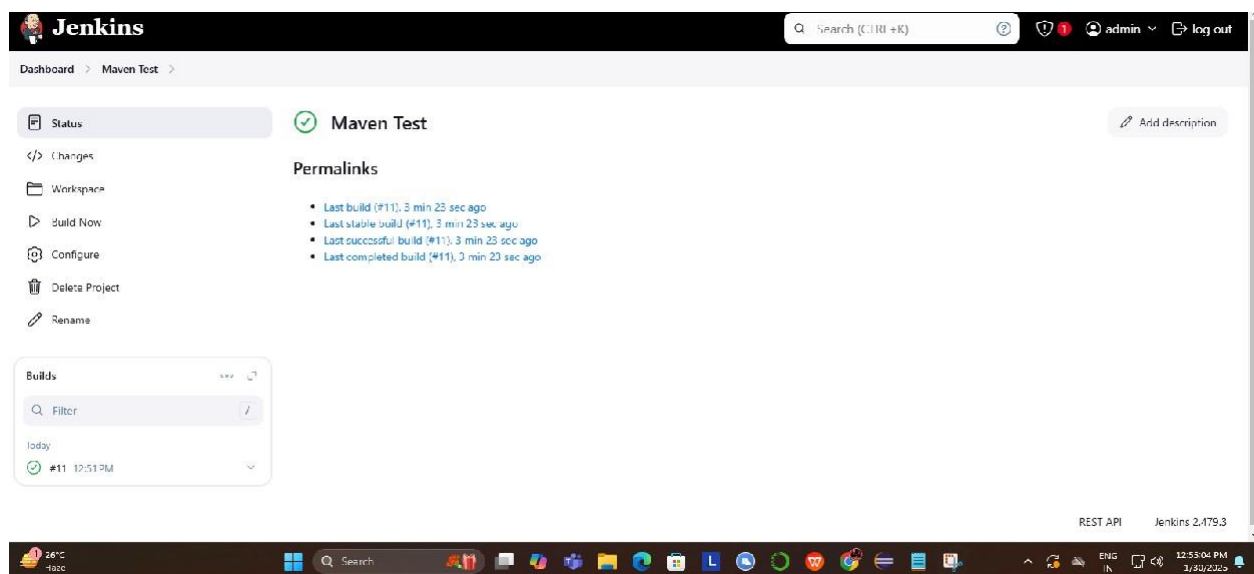
Enter Goal as **clean install**

Before you save and apply just below Goal there is Advance option add pom.xml path

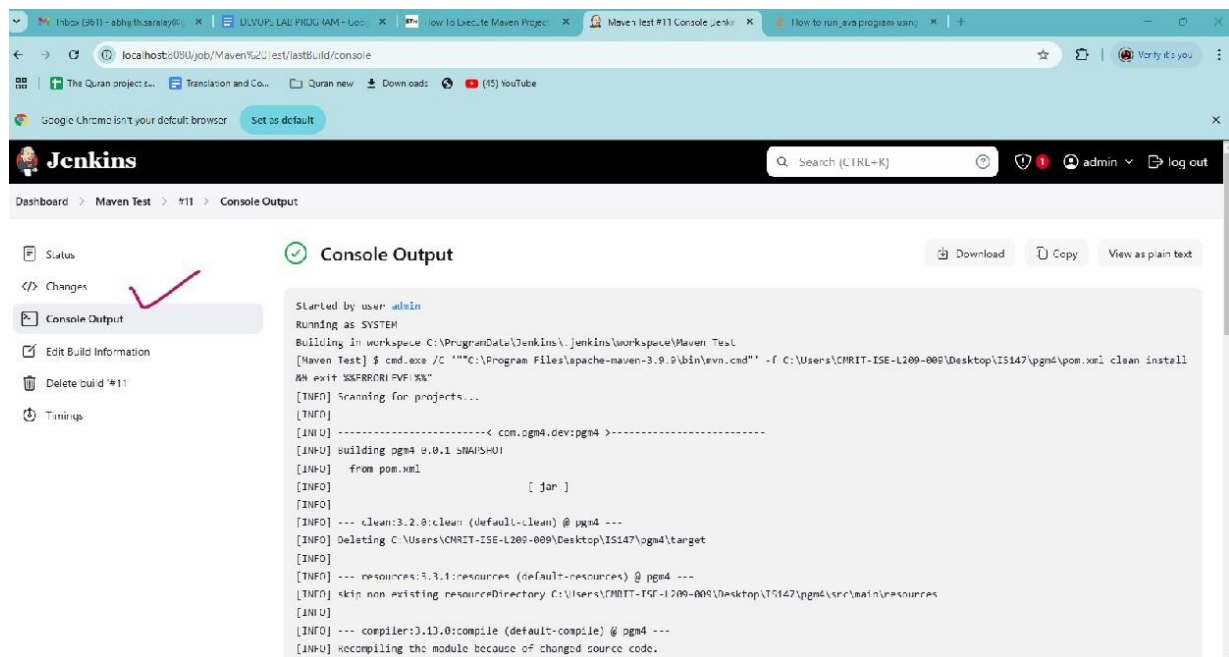
Goto pom.xml of your particular pgm and take path in my case its **C:\Users\CMRIT-ISE-L209-**

009\Desktop\IS147\pgm4\pom.xml.

After all Steps is over click on **Build** button the output be as in below



To see either click on build texts and goto console output or u can goto dashboard and opt to see build scripts



Program7: Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks, and Modules, Automating Server Configurations with Playbooks, Hands-On: Writing and Running a Basic Playbook

What is Ansible?

Ansible is an open-source IT automation and configuration management tool. It allows you to manage multiple servers and perform tasks such as:

- **Configuration Management:** Automate the configuration of servers.
- **Application Deployment:** Deploy applications consistently.
- **Orchestration:** Coordinate complex IT workflows and processes.

Key Concepts in Ansible

Inventory: An inventory is a file (usually in INI or YAML format) that lists the hosts (or groups of hosts) you want to manage. It tells Ansible which machines to target.

Playbook: A playbook is a YAML file that defines a set of tasks to be executed on your target hosts. It is the heart of Ansible automation. In a playbook, you specify:

- **Hosts:** The target machines (or groups) on which the tasks should run.
- **Tasks:** A list of actions (using modules) that should be executed.
- **Modules:** Reusable, standalone scripts that perform specific actions.

Modules: Ansible comes with a large collection of built-in modules (such as `apt`, `yum`, `copy`, `service`, etc.). These modules perform specific tasks on target hosts. You can also write custom modules if needed.

Why Use Ansible?

- **Agentless:** Ansible uses SSH to communicate with target hosts, so no agent needs to be installed on them.
- **Simplicity:** Playbooks use simple YAML syntax, making them easy to write and understand.
- **Idempotence:** Ansible tasks are idempotent, meaning running the same playbook multiple times yields the same result, ensuring consistency.
- **Scalability:** Ansible can manage a small number of servers to large infrastructures with hundreds or thousands of nodes.

Step 1: Create a Directory for Your Ansible Files

To keep your Ansible files organized, create a new directory for the project.

- Open your terminal. Run the following command to create a new directory and navigate to it:

```
mkdir ~/ansible-demo
```

```
cd ~/ansible-demo
```

Step 2: Create the Inventory File (inventory.ini)

The inventory file defines the target machine(s) for your Ansible tasks. In this case, we will run the playbook locally on the same machine.

- Create a new file named inventory.ini:

```
nano inventory.ini
```

- In the inventory.ini file, add the following content to specify localhost as your target machine:

```
[local]  
localhost ansible_connection=local
```

- **[local]:** This is the group name. You can specify more hosts under this group if needed.
- **localhost ansible_connection=local:** This tells Ansible to run the playbook locally without using SSH.

Step 3: Create the Playbook File (install_nginx.yml)

Now you'll create a playbook that installs and starts the Nginx service.

1. Create a new file named install_nginx.yml:

```
nano install_nginx.yml
```

2. Inside the file, add the following content:

```
- name: Install Nginx on Localhost  
hosts: local  
become: yes  
tasks:  
- name: Install Nginx  
apt:  
name: nginx
```


state: present

- name: Start Nginx Service

service:

name: nginx

state: started

enabled: yes

- **hosts: local** - This tells Ansible to run the playbook on the localhost, as defined in the inventory file.
- **become: yes** - This allows Ansible to run tasks as a superuser (using sudo).
- **apt:** - This module installs Nginx. The state: present ensures it's installed.
- **service:** - This module starts the Nginx service and ensures it's enabled to start on boot

Step 4: Run the Playbook Locally

Now that your playbook and inventory files are ready, you can run the playbook to install and start Nginx.

3. To run the playbook, execute the following command:

sudo ansible-playbook -i inventory.ini install_nginx.yml

- **-i inventory.ini** - Specifies the inventory file where the target machine(s) are defined.
- **install_nginx.yml** - Specifies the playbook file you want to run.

Ansible will connect to the localhost and execute the tasks defined in your playbook (installing Nginx and starting the service).

Step 5: Verify the Installation

After running the playbook, you can verify that Nginx was installed and is running correctly.

Option 1: Check in a Web Browser

Open a web browser and go to <http://localhost>.

You should see the default Nginx welcome page if the installation was successful.

Option 2: Check the Status of Nginx Service

4. Alternatively, check the status of the Nginx service by running the following command in your terminal:

systemctl status nginx

This command will show whether the Nginx service is active and running.

Program 8: Practical Exercise: Set Up a Jenkins CI Pipeline for a Maven Project, Use Ansible to Deploy Artifacts Generated by Jenkins

A. Setup git account

Step 1: Login to windows . Setup git account

Copy maven project to git repository(From eclipse)

```
git init
git add .
git commit -m "Initial commit of HelloMaven project"
```

Open pom.xml file and add the Code in the respective tags:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.mavenproject10</groupId>
  <artifactId>project10</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>project10</name>
  <url>http://maven.apache.org</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
```

```

<!-- Build section: defines how the project should be built -->
<build>
  <plugins>
    <!-- Plugin for creating executable JAR files -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-jar-plugin</artifactId>
      <version>3.2.2</version>
      <configuration>
        <archive>
          <manifest>
            <mainClass>com.mavenproject10.project10.App</mainClass> <!-- ₹ Change to your
actual main class -->
          </manifest>
        </archive>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>

```

B. Ansible Deployment in Jenkins

Step 1: Writing an Ansible Playbook for Deployment

Create an Ansible playbook that deploys the Maven artifact (the JAR file) generated by Jenkins to a target directory.

Step1: Create an Inventory File

- Open Your Text Editor and create a file called deploy.yml :
nano host.ini

- Enter the Following ini Code:
[local]
localhost ansible_connection=local

Step 2:. Create the Deployment Playbook

- Open Your Text Editor and create a file called deploy.yml :
nano deploy.yml
- Enter the Following YAML Code:

```

--
- name: Deploy Maven Artifact
  hosts: local
  become: yes
  tasks:

    - name: Copy the artifact to the deployment directory
      copy:
        src:"/var/lib/jenkins/workspace/ansible-demo/target/HelloMaven-1.0-SNAPSHOT.jar"

```

dest: "/opt/deployment/HelloMaven.jar"

C. Creating of new job in jenkins

Step1: Login to ubuntu

(Install Jenkins in ubuntu)

Loginto Jenkins

http://localhost:8080

Step2. Create new job

Create a New Job:

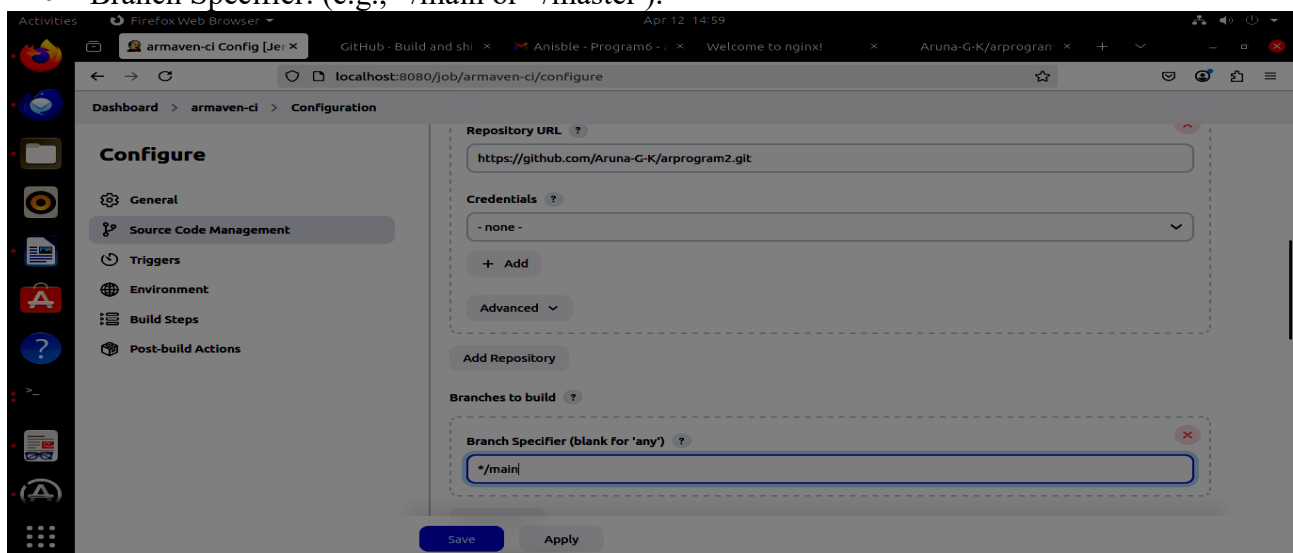
- Click “New Item” on the Jenkins dashboard.
- Enter an Item Name: e.g., HelloMaven-CI .
- Select “Freestyle project” and click “OK”.

Step3: Scroll to the “Source Code Management” Section:

- Select “Git”.
- Repository

Step4 :URL:

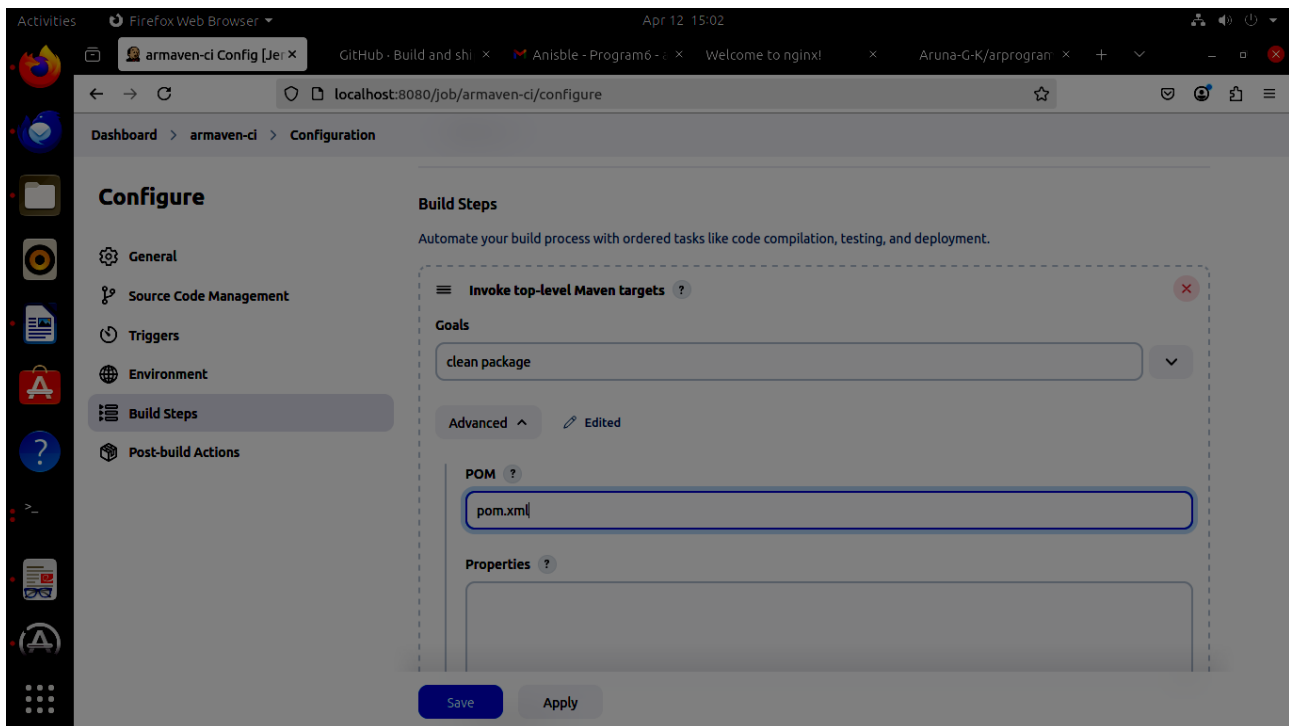
- Enter your repository URL
(e.g., <https://github.com/yourusername/HelloMaven.git>).
- Branch Specifier: (e.g., */main or */master).



Step 5: Add a Maven Build Step

- Scroll Down to the “Build” Section:

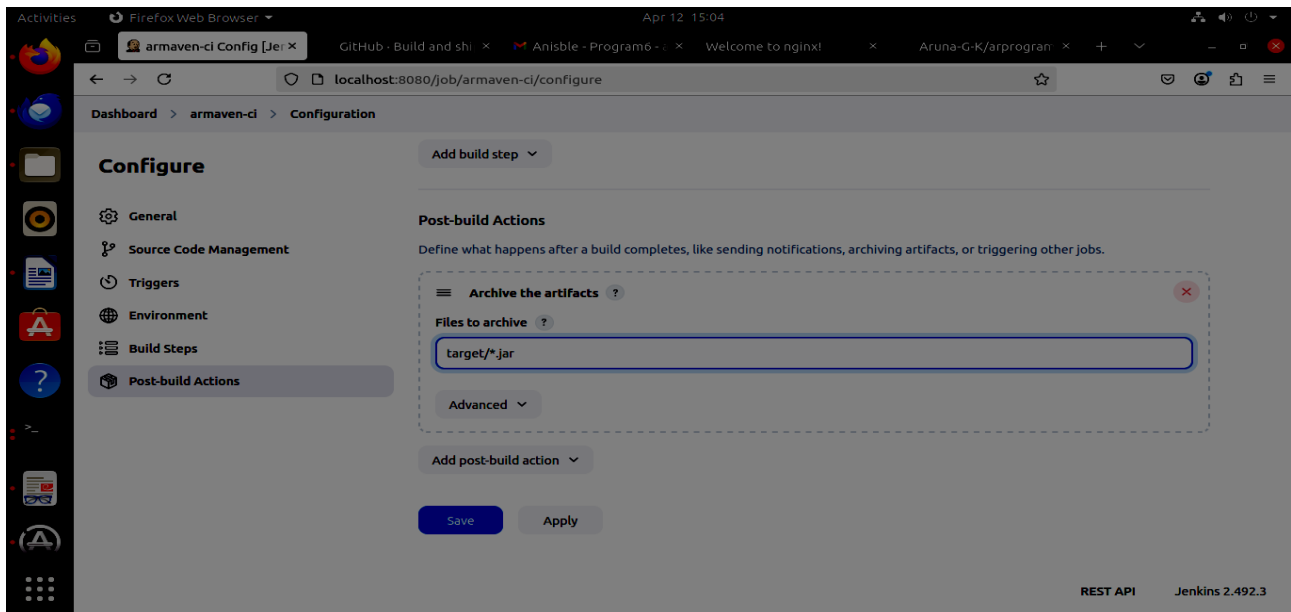
- Click “Add build step” and select “Invoke top-level Maven targets”.
- Configure the Maven Build:
 - Goals: Type: **clean package**



Step 6: Archiving the Artifact

- After the Maven build completes, you need to archive the generated artifact so that it can be used later by the deployment process.
 - Scroll Down to the “Post-build Actions” Section:
 - Click “Add post-build action” and select “Archive the artifacts”.
- Configure Artifact Archiving:

- Files to Archive: Type: target/*.jar



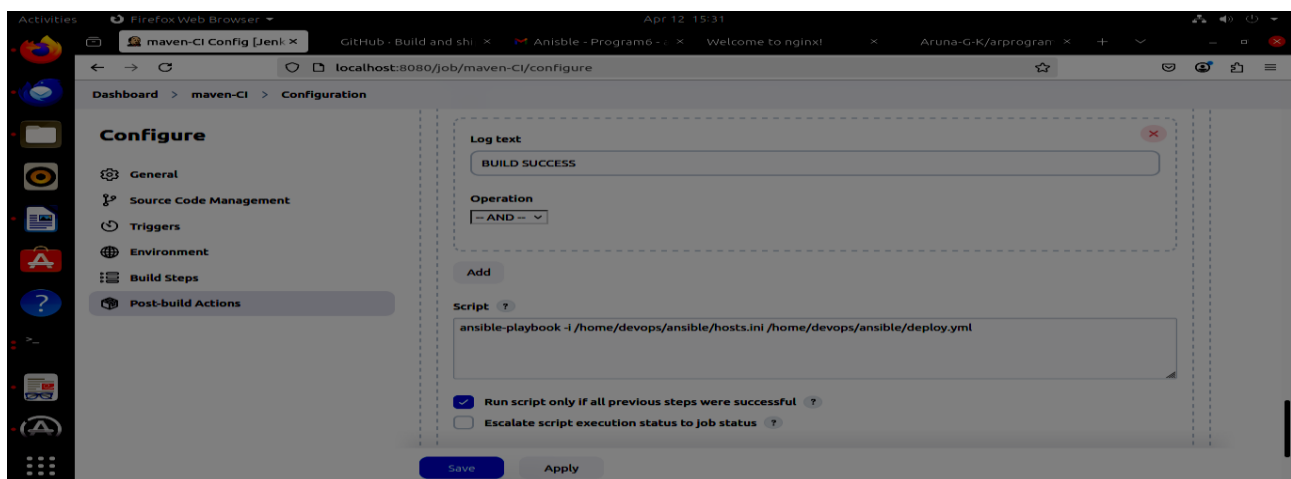
- Click “Add post-build action” and select “post build task”.
 - Go to Script and Type the following command:

ansible-playbook -i /home/devops/ansible-demo/host.ini /home/devops/ansible-demo/deploy.yml

ansible-demo → directory Name

host.ini → Name of the Inventory file in ansible where host is mentioned.

deploy.yml → name of the yaml file in ansible where task is mentioned.



Step7: Click on save and Build now



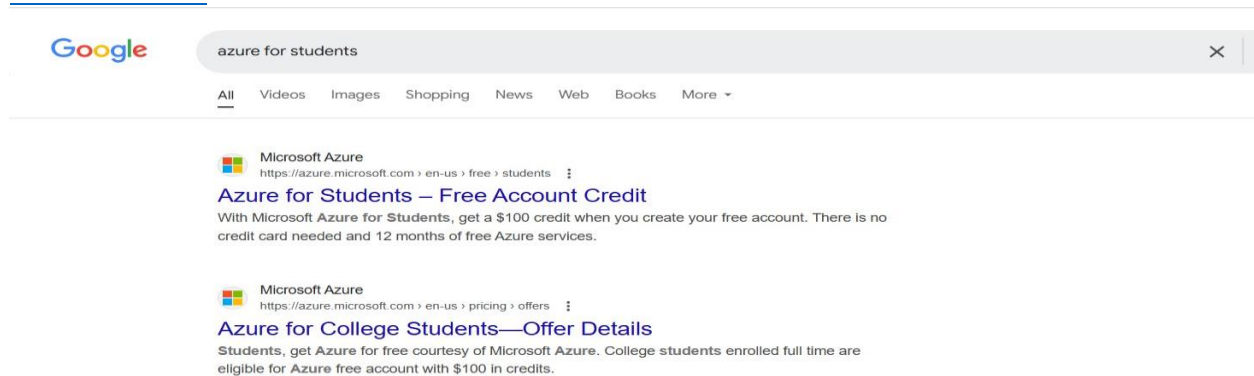
Step8: Go to directory that you have mentioned in ansible yml file(ansible-demo) and check for jar file and open the jar file and execute the snapshot using terminal.

Step9: Following output will be displayed in terminal screen.

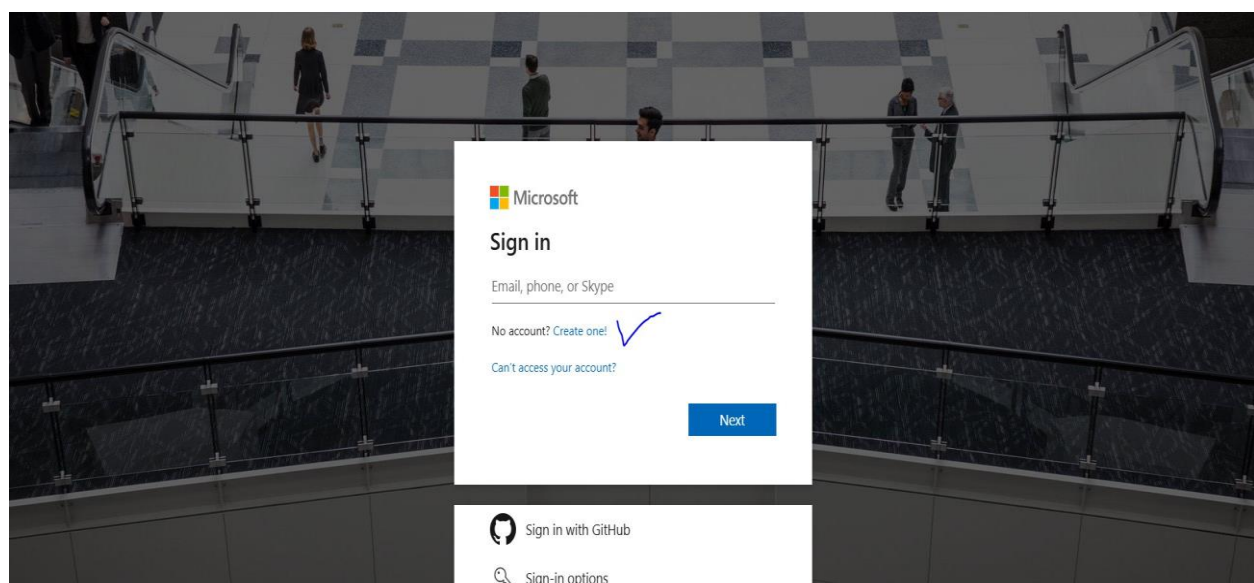
Hello world

Program 10: Introduction to Azure DevOps: Overview of Azure DevOps Services, Setting Up an Azure DevOps Account and Project.

STEP 1:Go to Google chrome and type **azure for students** <https://azure.microsoft.com/en-us/free/students>.

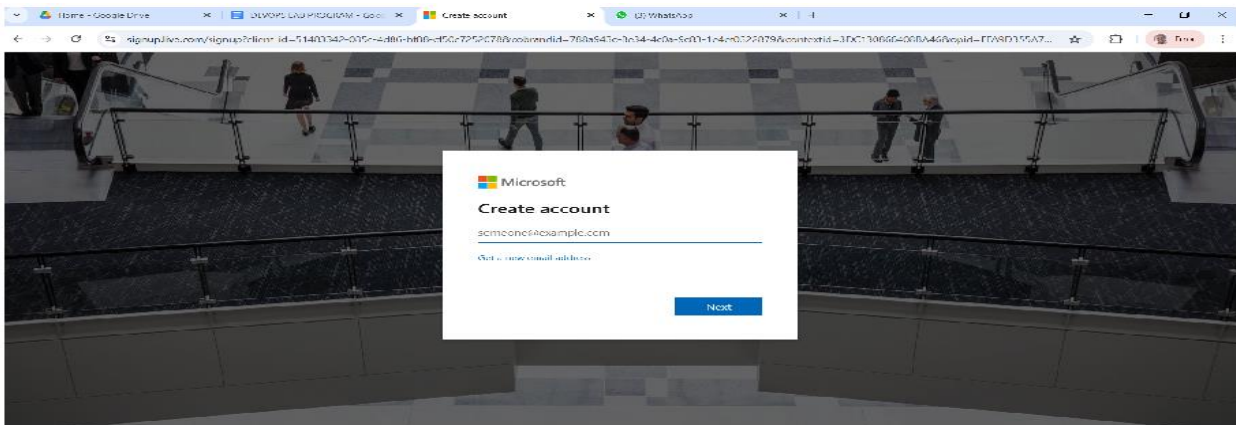


STEP 2: Click on **Start Free** after that u get screen as in below

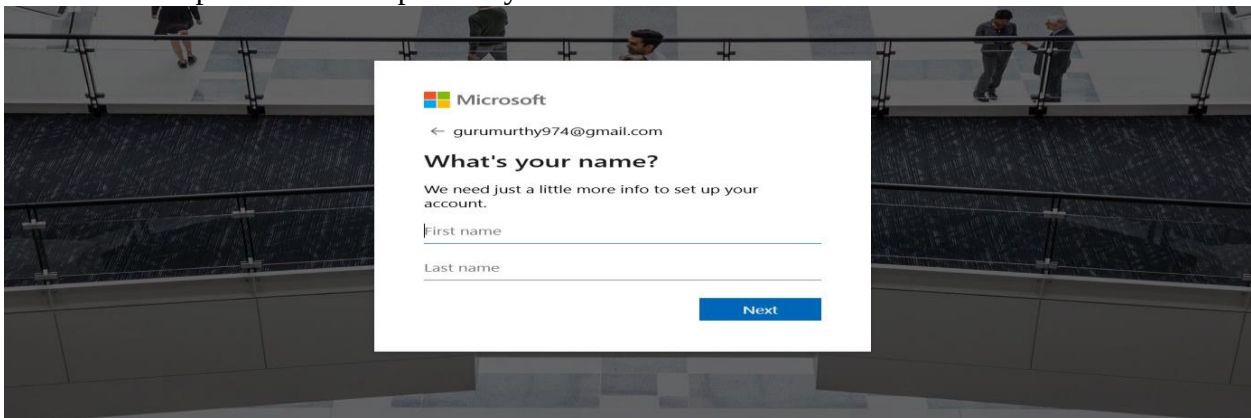


Click on Create one, If u have Github account u can Sign in using github account better way is to create one account

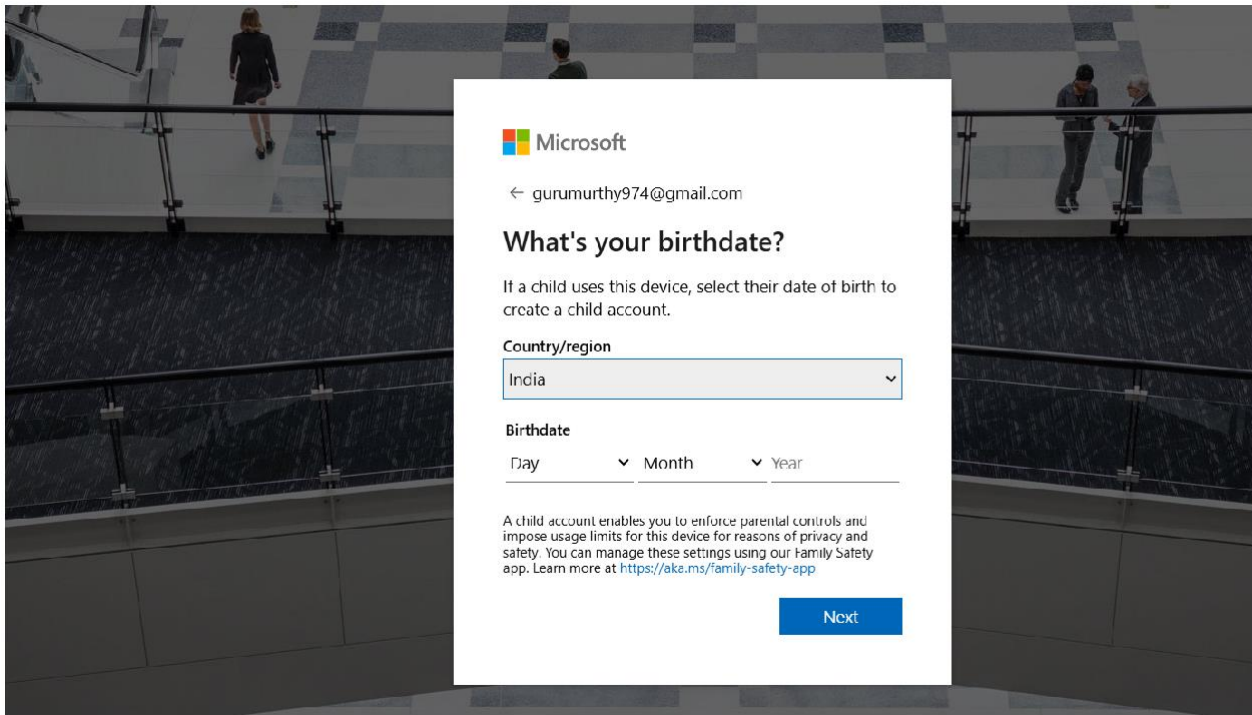
STEP 3: Provide your email id at place of create account



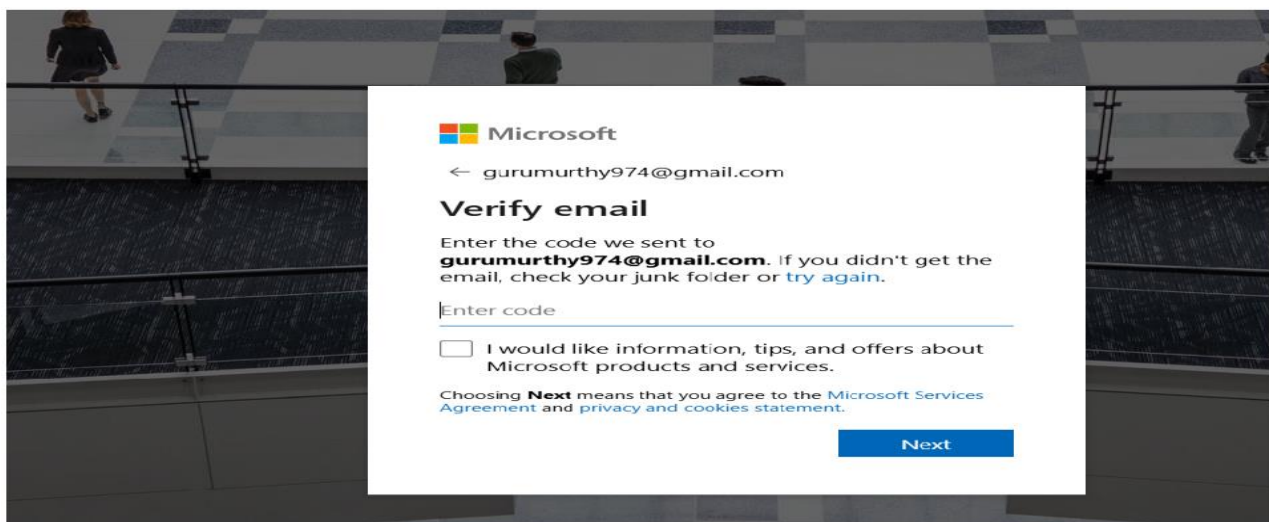
STEP 4: After password is set provide your First name Last name



Then provide Country, Date of Birth



STEP 5: Verification code be mailed to the mentioned once kindly type it.



STEP 6: After code is verified as u got in the mail referred U be given an option to solve puzzle game



← gurunurthysar974@gmail.com

Help us beat the robots

Solve a puzzle so we know you're not a robot.



Solve
puzzle



STEP 7: This step is the important once where u fill your Academic Details properly where u provide College email id as in provided by your Individual colleges later the verification code again comes

← → G signup.azure.com/studentverification/offer?type=1&correlationid=0697618e-7d2a-47db-95ac-185255fe0055 ☆ ⓘ Error

Microsoft Azure gurunurthysar974@gmail.com Sign out

Academic Verification

Start by entering your name as per the school records. Select your school's country and enter your school's name. Enter your date of birth as per the school records. The email address may be used to reach you if we have trouble verifying your application, so please enter your school provided email address.

First name

Last name

Country

If your country is not listed, the offer is not available in your region. [Learn More](#)

School name

School name will help provide Microsoft with additional information for verification. If available, please enter it here.

Date of birth

Start Chat

After proper college email is given u get verification mail with link to mail u have provided

Hello,

You have received this email because you recently requested verification via **Microsoft's Academic Verification** service. If you did not submit your email for this program, please disregard this email.

To complete your academic status verification, please click the link below. The link will automatically expire if not used within 5 days. After clicking the link, your verification status will be confirmed and you will return to the site.

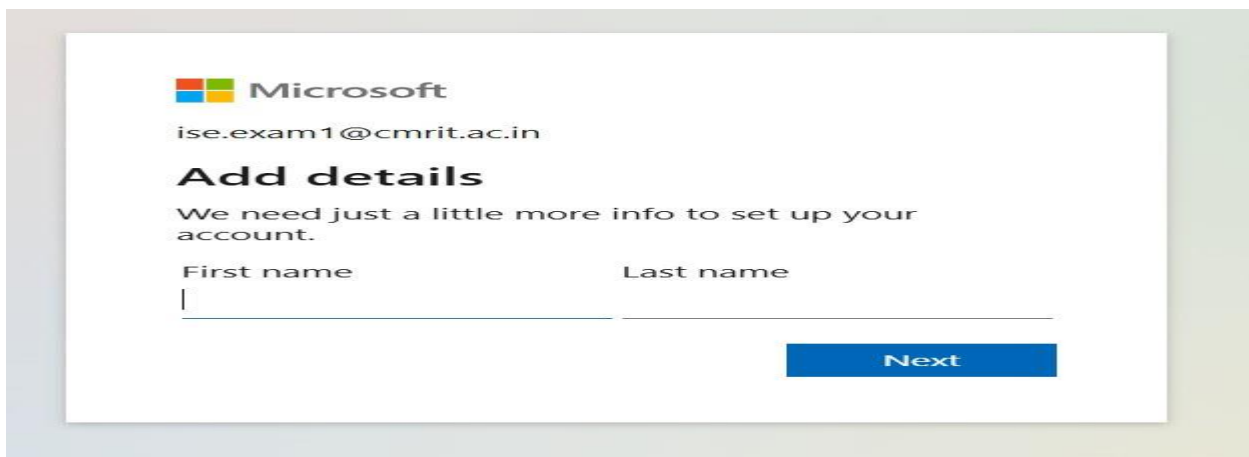
-Navigate to: <https://verifyemail.microsoft.com/v1.0/tokenverification/verify?signature=YPQldNoTloPJ8nJy8QXZhbyR5Y5wAfwKVaNqgX5hgX6Yb0hOyqW5Aa5CsLqDGTWbHGIFSKjx37ROJFB7B0BoqKqDTCsJ3ADCIXL9McGPZs97Ms2qbP54fx0AGGTgdu7Y2SGVdpLBk98PIP33rVZf%2F2ES3z4IR8zM%2BjjuRvGxNuTK%2B4IM2tfFKm2kSsbp%2BI754BLIZSJW4216NcWohhks8!%2Fdd2qk7fgLurMrUhi8MaiPwQjLzy8mWbd9bjRY4PtX%2FfhwPOTPGdROYFUppsYuf6vwYojSylbtFBATxAl1se5JlzZsAL2qdHwV03x8Yhm3gJN4TGYdxizDLE0CGCn6h8GVjyls%2BQEI096N0Rvqi%2FnlccNlbsdcmTAN0u9ivHcdpoHRnwX22Q58JgggCzyefDnDbFgxuiteaHQn%2BPKgKnOs26OHSdeVVEc%2B0MBsBAQjBTFiHeL7XEkydahHXReTh8a77XxbRAQAdXGXIfW0caHpUHafaNKxiQBm28dR%2BPgZB0n3kKFMIR20ltYQtxpSkOGa6DOssh38bumS2WrSQ%3D>

Thank You,

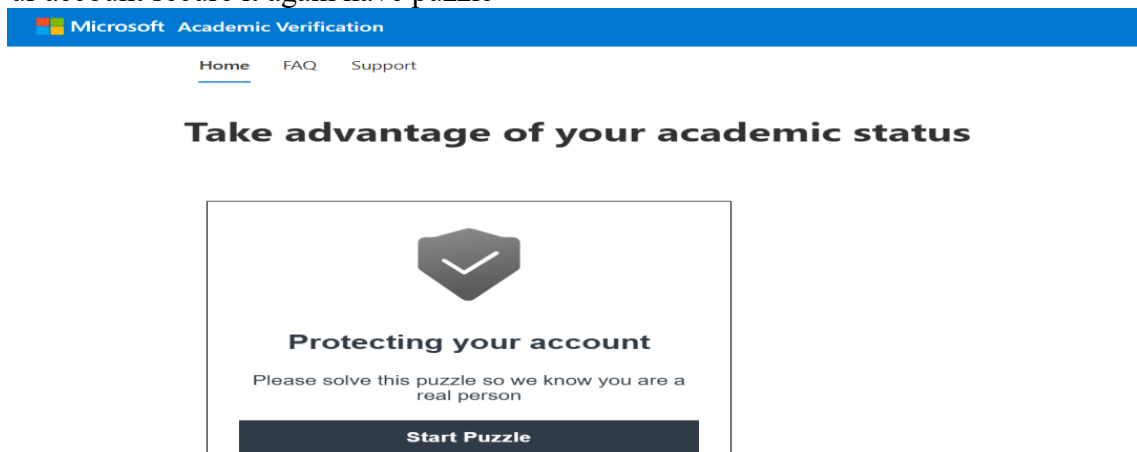
The **Microsoft Academic Verification Team**

Click on link sent

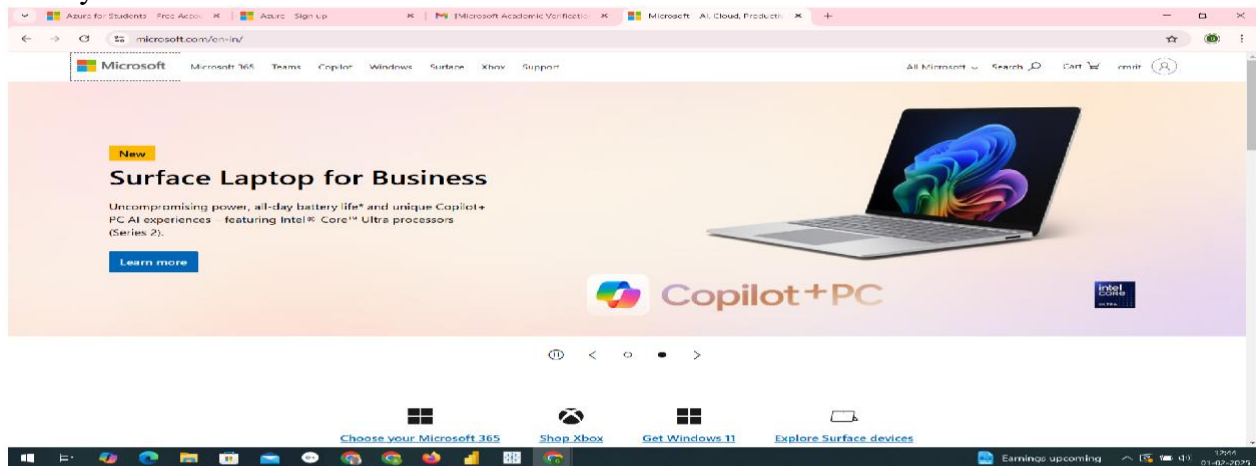
After u Click The screen be as in Below



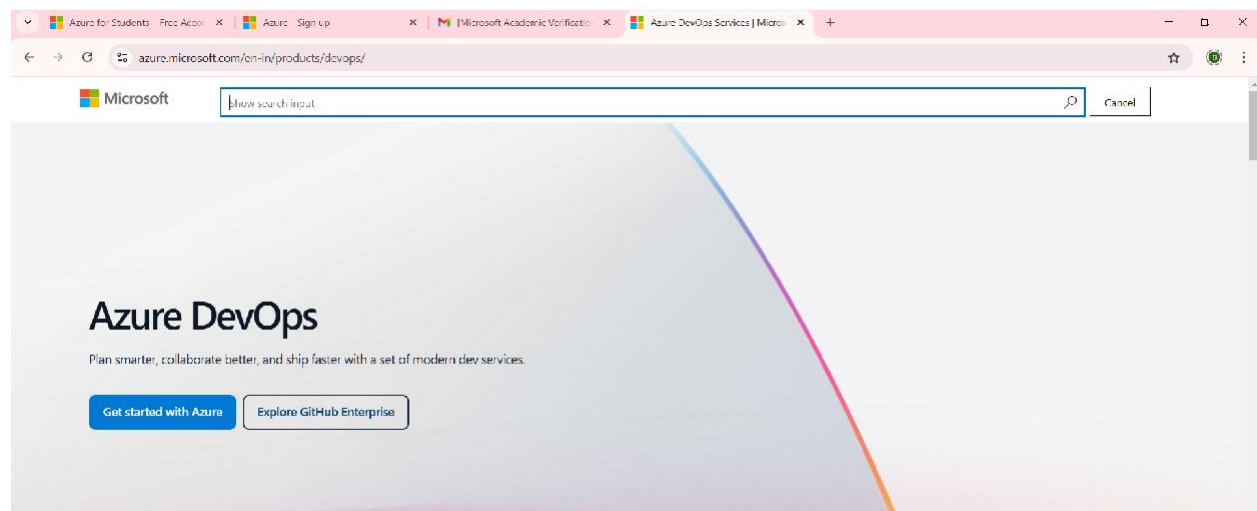
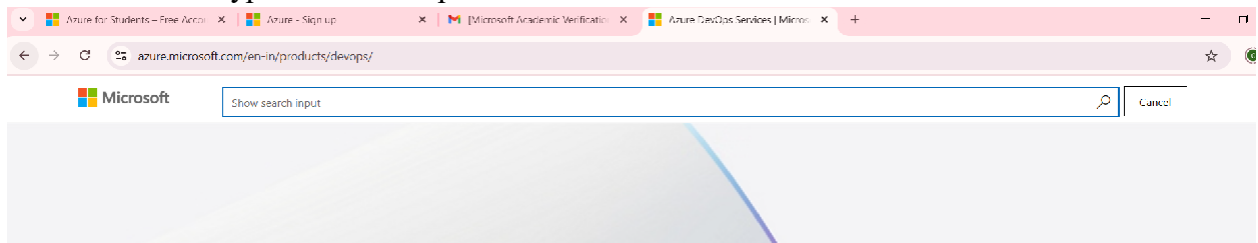
To make ur account secure it again have puzzle



Finally screen be like this

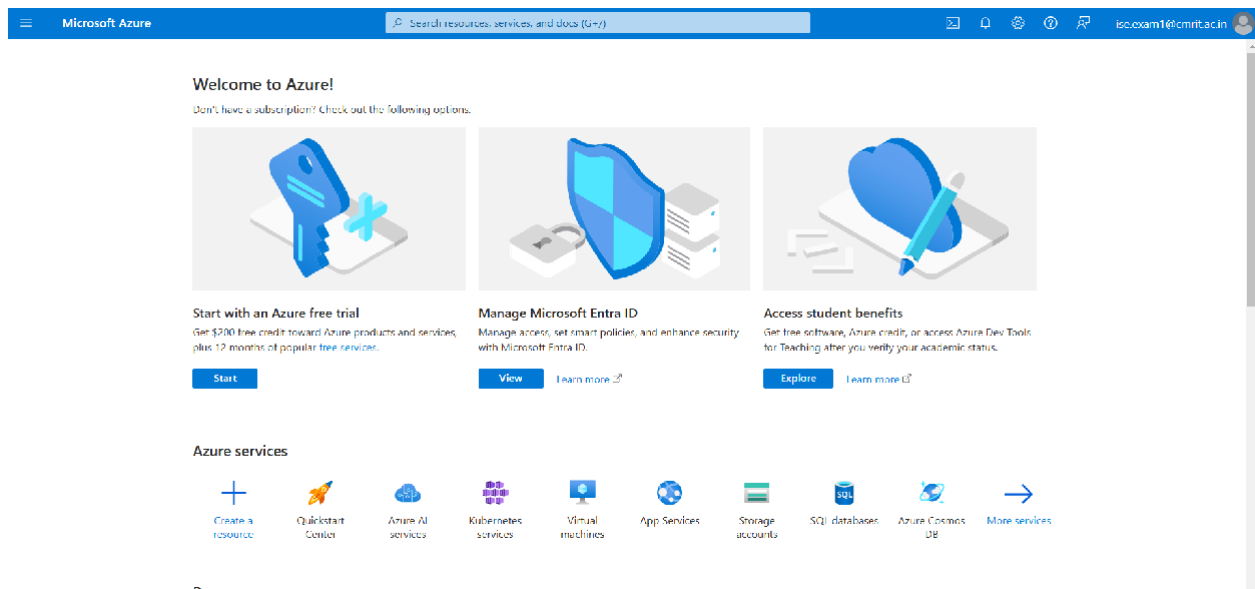


Go To search and type Azure DevOps

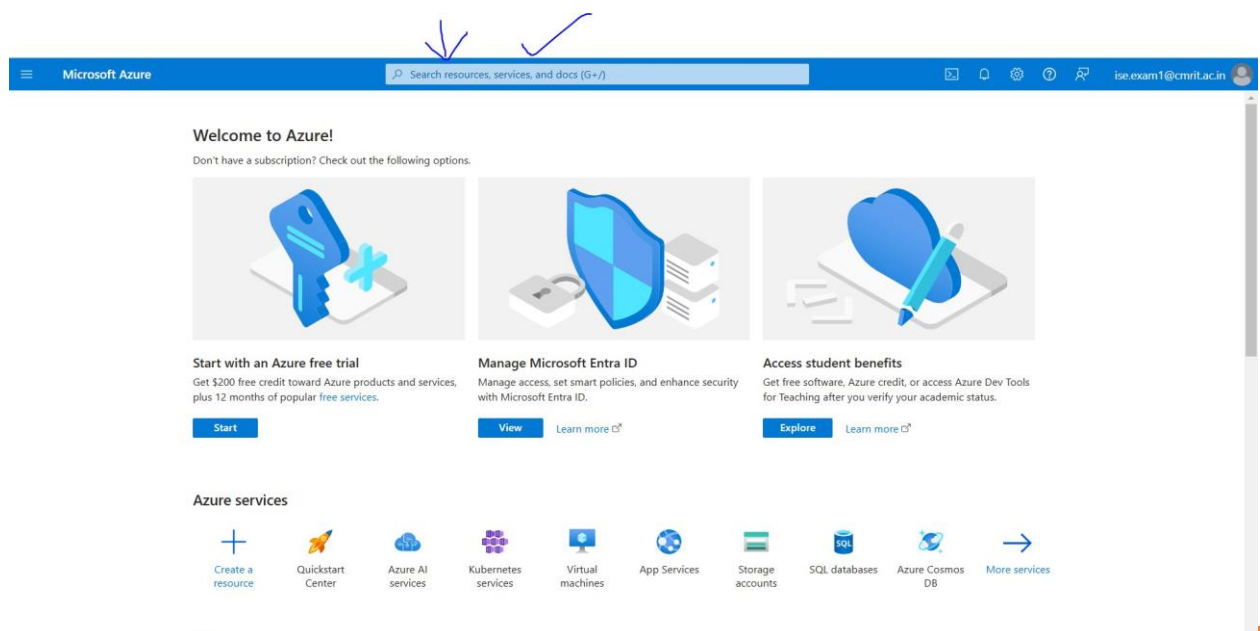


Click on Get started with Azure

After the click u get the screen of Get free need not to do anything just click on Signin You will get screen as in below

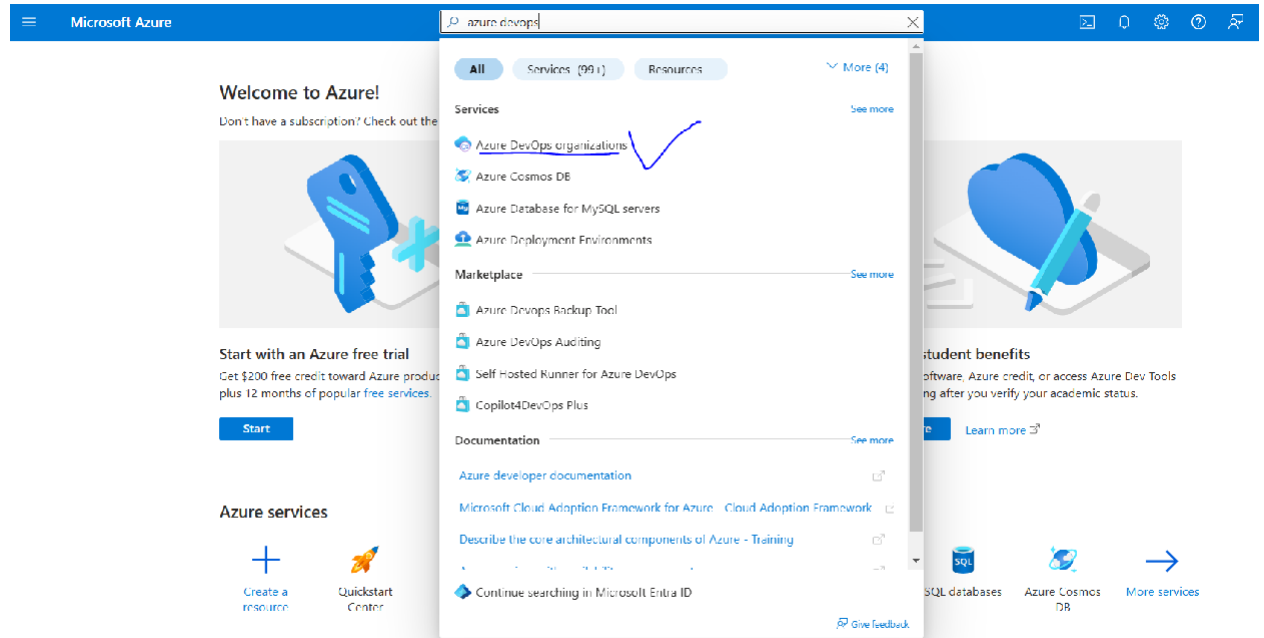


THIS IS THE HOME PAGE OF THE MICROSOFT AZURE where you can see number of services.
In top where have search for services

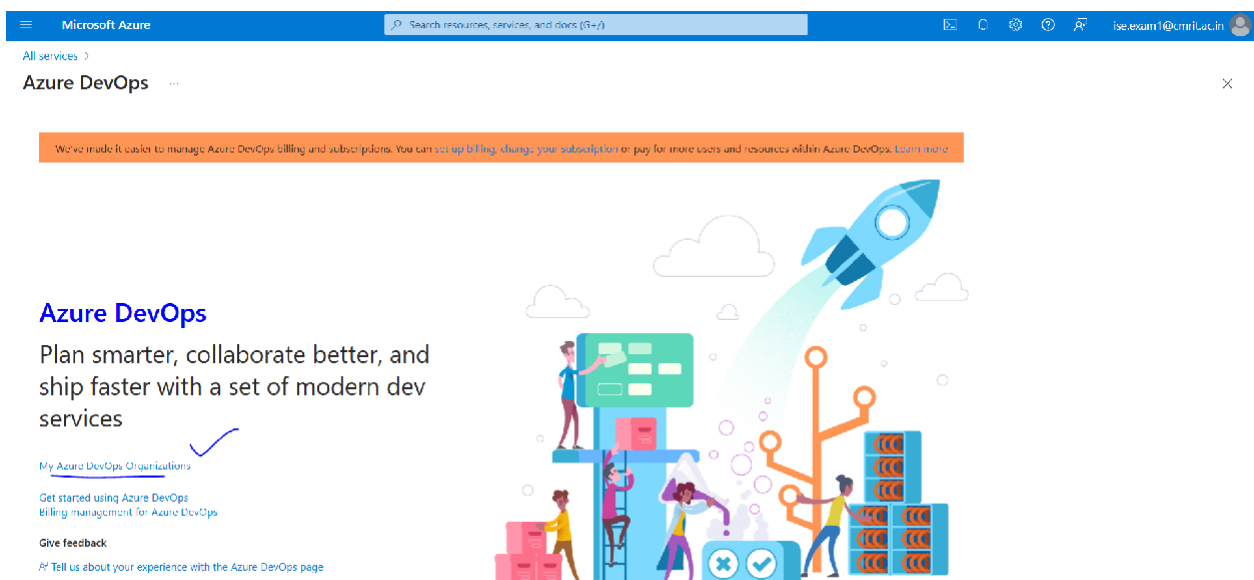


Type Azure DevOps

STEP 8: Select Azure DevOps Organization



STEP 9: After u opting for Azure DevOps Organizations u get a screen as in below now
Select **My Azure DevOps Organization**



After above selection it once again re verifies name and email just click **Continue** After it U get a Screen.

we

Azure DevOps Organizations

dev.azure.com/iseexam10557 (Owner)

Create a Team Project and start collaborating with your team now!

New project ✓

Actions

Open in Visual Studio

Finally After Creating a New Organization

U can create Project of ur choice as per requirement

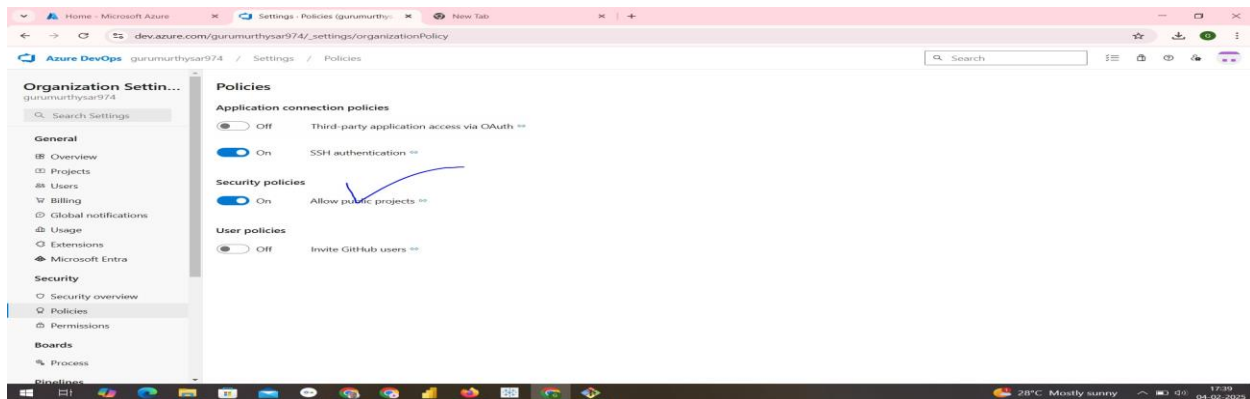
Every time u need not to signin u can bookmark or add the below link as shortcut

<https://aex.dev.azure.com/>

<https://portal.azure.com/#home>

PROGRAM10: Creating Build Pipelines: Building a Maven/Gradle Project with Azure Pipelines, Integrating Code Repositories (e.g., GitHub, Azure Repos), Running Unit Tests and Generating Reports.

STEP1: On creating organization goto Organization settings goto Policy And Allow Public Projects active



STEP3: Create Directory and create simple hello world maven project In Eclipse.

STEP4: to add files from local to github Follow the procedure

First create a repository in github as maventestazure

Then come to gitbash and type

git init

git add .

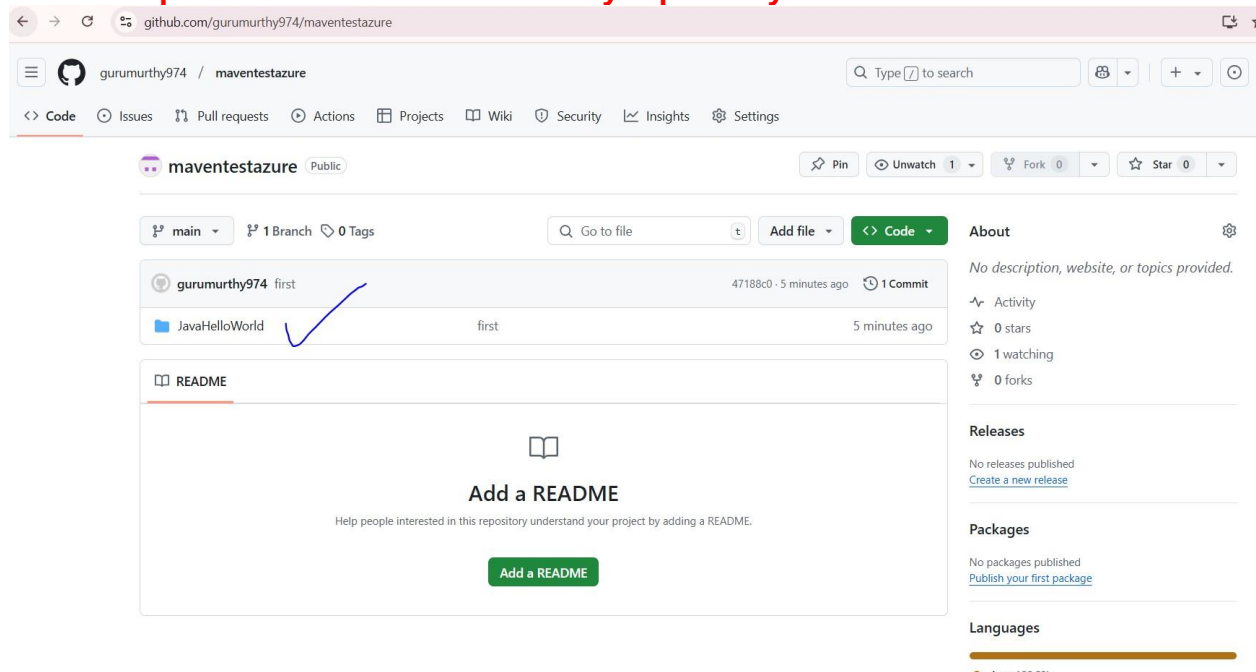
git commit -m "azure pipeline example" git

branch -M main

git remote add origin https://github.com/sushma-vittal-p/maventestazure1 (Git Repository Name)

git push -u origin main

After completion of above command my repository looks



STEP5: Now goto Azure DevOps Organization create Public Project

The first screenshot shows the 'Create a project to get started' page in Azure DevOps. The 'Project name' field is filled with 'maventestaz'. The 'Visibility' dropdown is set to 'Public'. The second screenshot shows the 'Welcome to the project!' page for 'maventestaz'. The 'Pipelines' tab is selected in the left sidebar. The third screenshot shows the 'Where is your code?' page, where 'GitHub' is selected as the source for the new pipeline.

STEP7: It asks for minimum signin verification after that ur screen be as in below select required repository there to run maven project in my case its **maventest123**

The screenshot shows the 'Select a repository' page in Azure DevOps. The 'Connect' tab is selected. A list of repositories is displayed, including 'gurunurthy974/maventestaz', 'gurunurthy974/ctol', 'gurunurthy974/repotest', and 'gurunurthy974/Maven'. The repository 'gurunurthy974/maventestaz' is highlighted.

Step8: After required repo is selected the screen be as in below

Your personal account

- Public profile
- Account
- Appearance
- Accessibility
- Notifications

Access

- Billing and plans
- Emails
- Password and authentication
- Sessions
- SSH and GPG keys
- Organizations
- Enterprises
- Moderation

Code, planning, and automation

<> Developer settings

Azure Pipelines

Installed 3 days ago · Developed by [microsoft](#) · <https://azure.microsoft.com/services/devops/pipelines/>

Continuously build, test, and deploy to any platform and cloud

[Azure Pipelines](#) offers cloud-hosted pipelines for Linux, macOS, and Windows with 10 free parallel jobs and unlimited minutes for open source projects.

- Any language, platform, and cloud**
Build, test, and deploy Node.js, Python, Java, PHP, Ruby, Go, C/C++, C#, Android, and iOS apps. Run in parallel on Linux, macOS, and Windows. Deploy to cloud providers like Azure, AWS, and GCP. Distribute mobile apps through beta channels and app stores.
- Native container support**
Create new containers with ease and push them to any registry. Deploy containers to independent hosts or Kubernetes.
- Advanced workflows and features**
Easy build chaining and multi-phased builds. Support for YAML, test integration, release gates, reporting, and more.
- Extensible**
Use a range of build, test, and deployment tasks built by the community – hundreds of extensions from Slack to SonarCloud. You can even deploy from other CI systems, like Jenkins. Webhooks and REST APIs help you integrate.

Only select repositories
Select at least one repository.
Also includes public repositories (read-only).

Select repositories ▾

Selected 1 repository.

gurunurthy974/maventestazure (suggested) ×

Approve and install Cancel

Danger zone

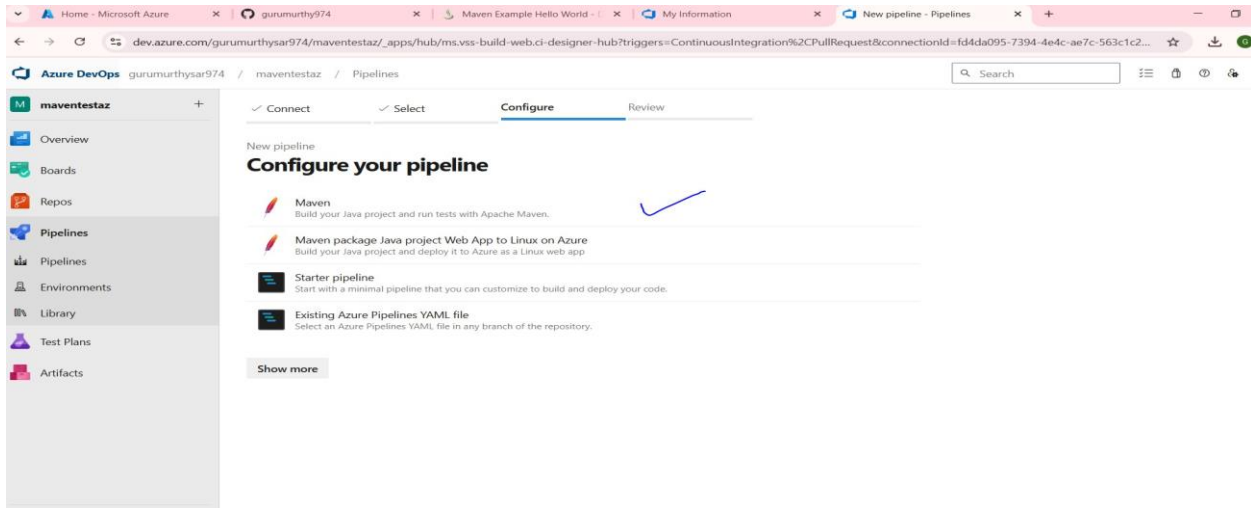
Suspend your installation
This will block the app access to your resources. [Suspend](#)

Uninstall "Azure Pipelines"
This will remove the app and revoke access to all resources. [Uninstall](#)

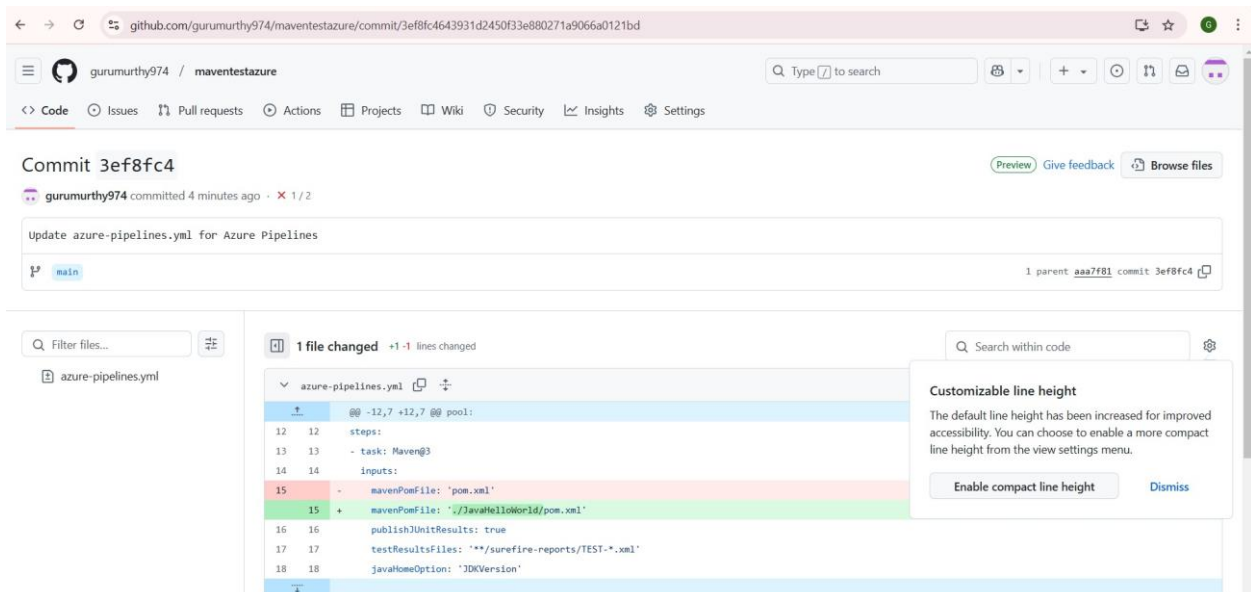
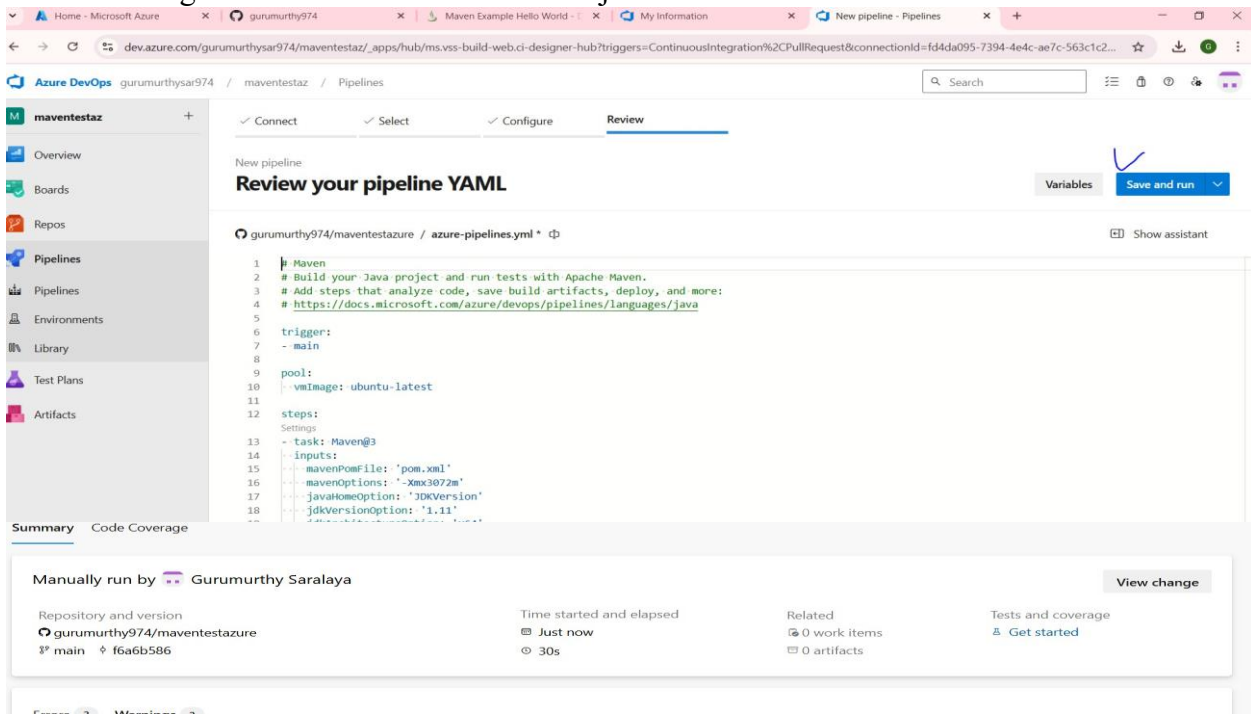
© 2025 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Approve

STEP 9: It again verifies signin verification of microsoft account You be able to see starter pipeline select for Maven

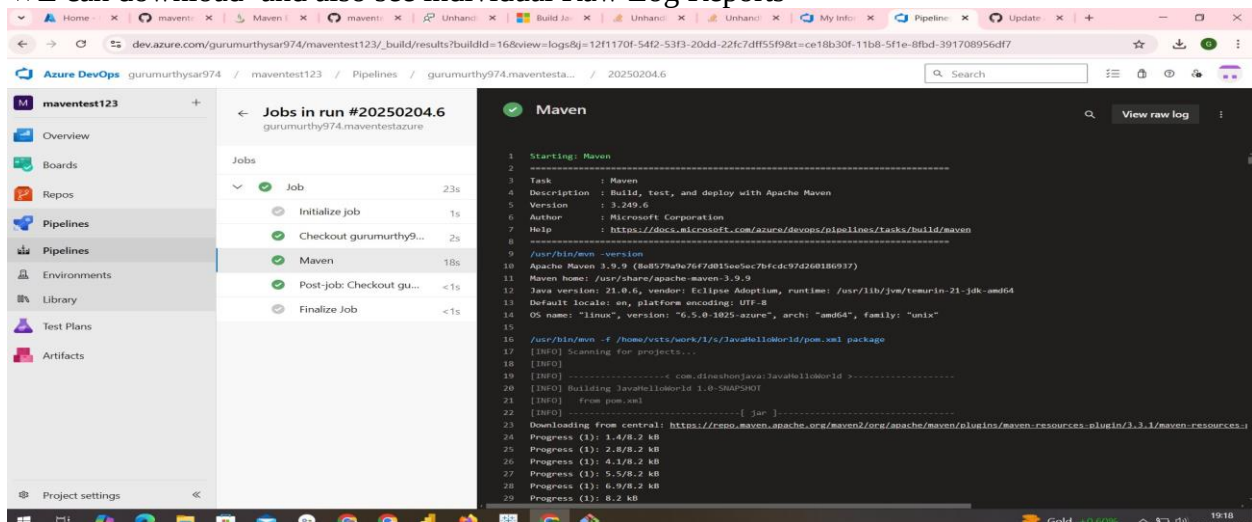


After selecting maven it asks for save and run just click on it



The commits will also be visible in github

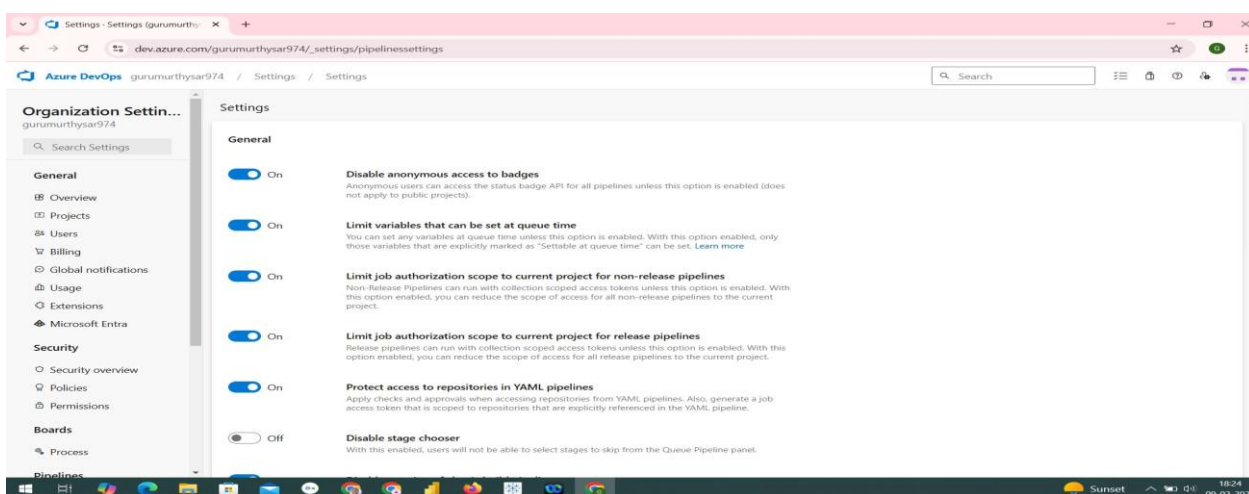
WE can download and also see individual Raw Log Reports



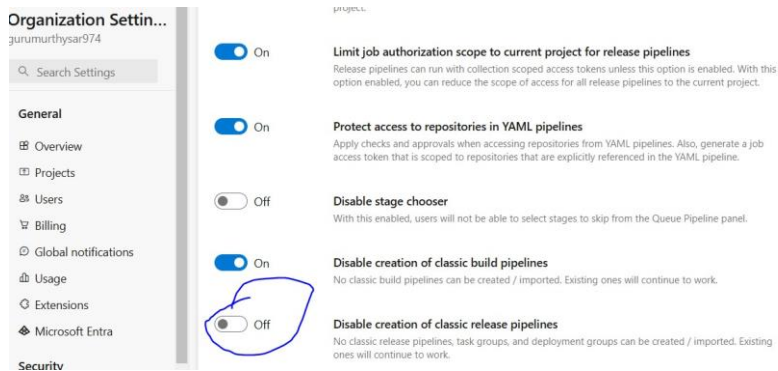
If pipeline permission error comes please complete the registration form of Parallel Jobs after 2 working days(48hrs) u be able to run pipelines for private projects same happens to be for public Projects.

Program11: Creating Release Pipelines: Deploying Applications to Azure App Services, Managing Secrets and Configuration with Azure Key Vault, Hands-On: Continuous Deployment with Azure Pipelines.

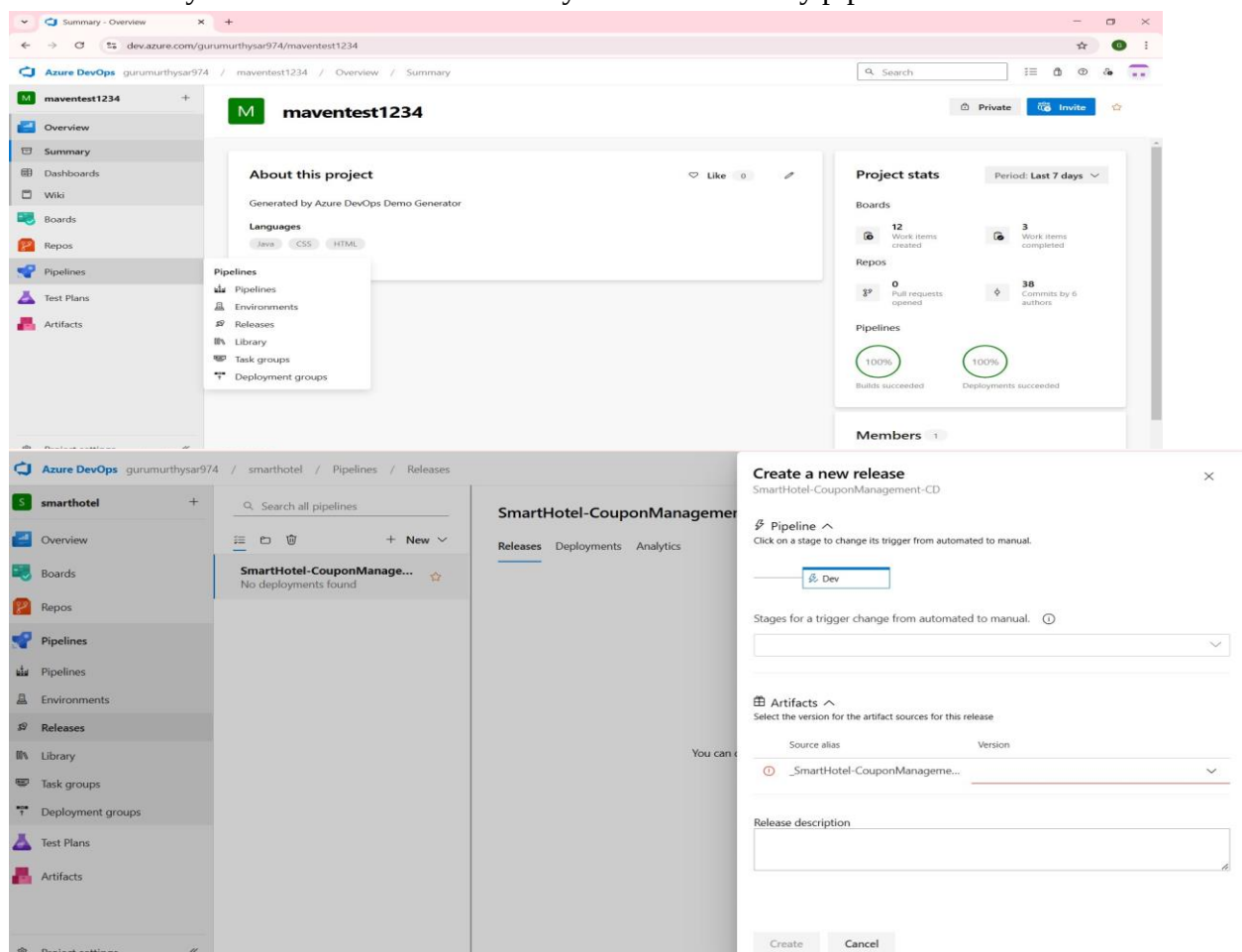
STEP1: Click on Organization setting and click on Pipeline Settings You get screen as in below



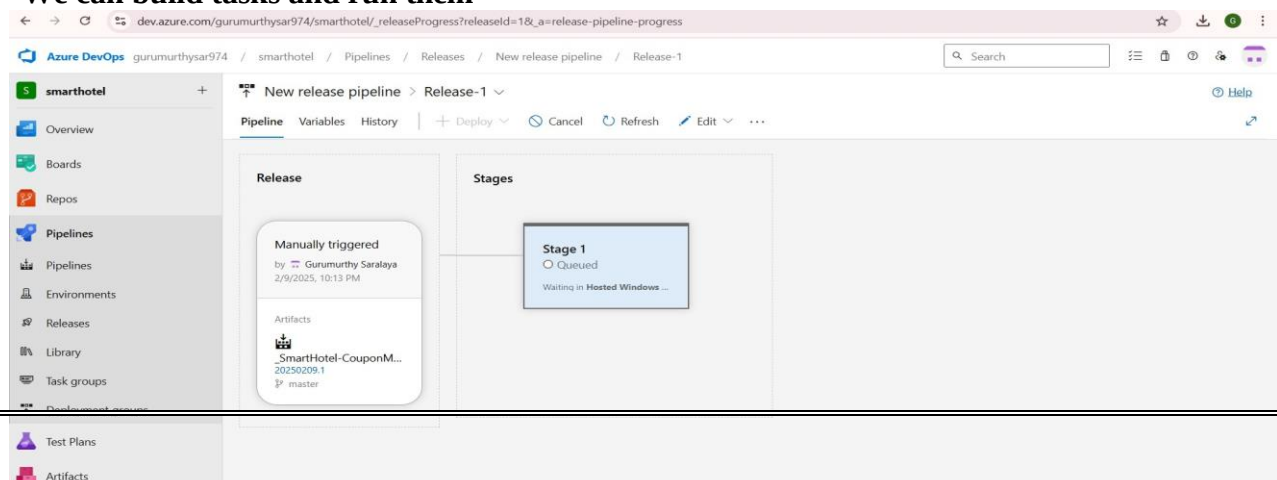
STEP2: Off the Disable creation of classic pipeline



STEP3: Now you be able to see the visibility of Release for any pipeline creation as in screen below.



We can build tasks and run them



New release pipeline > Release-3 > Stage 2 ✓ Succeeded

← Pipeline Tasks Variables **Logs** Tests | [Deploy](#) [Cancel](#) [Refresh](#) [Download all logs](#) [Edit](#) ...

Deployment process
succeeded

Agent job
Succeeded · 1 warning

Agent job Started: 2/9/2025, 10:52:10 PM ... 44s
Pool: Hosted Windows 2019 with ... · Agent: Hosted Agent

✓ Initialize job · succeeded	7s
✓ Download Artifacts · succeeded	4s
✓ Maven D:\a\r1a_gurumurthy974_maventestazure/JavaHelloWorld/pom.xml · succeeded 1 warning	32s
✓ Finalize Job · succeeded	<1s

dev.azure.com/gurumurthysar974/smarthotel/_releaseProgress?_a=release-pipeline-progress&releaseId=5

Azure DevOps gurumurthysar974 / smarthotel / Pipelines / Releases / New release pipeline / Release-3

New release pipeline > Release-3

Pipeline Variables History | [Deploy](#) [Cancel](#) [Refresh](#) [Edit](#) ...

Release

Manually triggered
by Gurumurthy Saralaya
2/9/2025, 10:51 PM

Artifacts
_gurumurthy974_mave...
3ef8fc464
main

Stages

Stage 2
✓ Succeeded
1 warning
on 2/9/2025, 10:52 PM
100%

Program12: Practical Exercise and Wrap-Up: Build and Deploy a Complete DevOps Pipeline, Discussion on Best Practices and Q&A

Overview

In this experiment, you will create an end-to-end DevOps pipeline that demonstrates the following processes:

- **Version Control:** Code is maintained in a Git repository (e.g., GitHub or Azure Repos).
- **Continuous Integration (CI):**
 - A CI tool (Jenkins and/or Azure Pipelines) automatically checks out the code, builds it using Maven/Gradle, runs unit tests, and archives the build artifact.
- **Artifact Management:** The artifact (e.g., a JAR file) is archived and made available for deployment.

- **Continuous Deployment (CD):**
 - Deployment automation is handled either by an Ansible playbook or an Azure Release pipeline, deploying the artifact to a target environment (such as an Azure App Service or a local server).
- **Secrets and Configuration Management:** Securely manage configuration details and secrets using Azure Key Vault.
- **Pipeline as Code:** Use YAML (for Azure Pipelines) or a Jenkinsfile (for Jenkins) to define your build and release processes.

After setting up and running the pipeline, we will discuss best practices for designing and maintaining such pipelines and open the floor for a Q&A discussion.

Prerequisites

Before starting, ensure you have completed or have access to the following:

- **Source Code Repository:**
 - A Java project (e.g., “HelloMaven” or “HelloGradle”) hosted on GitHub or Azure Repos.
 - The project should follow a standard structure (with pom.xml for Maven or build.gradle for Gradle, and appropriate src/main/java and src/test/java directories).
- **Jenkins and/or Azure DevOps Setup:**
 - Jenkins installed and configured on your local machine or a cloud server (refer to Experiments 5 and 6), or an Azure DevOps project set up with a build pipeline (Experiments 9 and 10).
- **Ansible Installed:**
 - Ansible is installed on your control machine (or Jenkins server) with a basic inventory file (see Experiment 7).
- **Azure Resources:** (Optional but recommended for cloud deployment)
 - An Azure App Service instance created to host your application.
 - An Azure Key Vault instance set up to store sensitive data (e.g., connection strings).
- **Access Credentials:**

- Permissions to commit code to the repository.
 - Administrative access on Jenkins/Azure DevOps.
 - Proper permissions on Azure to deploy to App Services and to manage Key Vault secrets.
- ### Step-by-Step Pipeline Setup

Step 1: Code Repository Preparation

1. Ensure Your Project Is in Version Control:

- Navigate to your project folder (e.g., “HelloMaven”) and initialize Git if not already done:
- `cd /path/to/HelloMaven` `git init` `git add .`
- `git commit -m "Initial commit of HelloMaven project"` Push the project to your remote repository (e.g., GitHub):
- `git remote add origin https://github.com/yourusername/HelloMaven.git` `git push -u origin main`

Step 2: Continuous Integration with Jenkins and/or Azure Pipelines

A. Jenkins CI Pipeline Setup

1. Create a New Jenkins Job:

- Log in to Jenkins and click “**New Item**”.
- Enter a name (e.g., HelloMaven-CI) and select “**Freestyle project**” or “**Pipeline**”.
- *Screenshot Tip:* Capture the new job creation screen.

2. Configure Source Code Management:

- Under the “**Source Code Management**” section, select “**Git**”.
- Enter your repository URL (e.g., `https://github.com/yourusername/HelloMaven.git`) and set branch specifier to `*/main`.

3. Add Build Steps:

For Maven Projects:

- Add a build step “Invoke top-level Maven targets” and set the goals to:
- `clean package`

For Pipeline-as-Code (Jenkinsfile):

Create a Jenkinsfile in your repository with stages for checkout, build, test, and archive.

For example:

```
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps {
                git url: 'https://github.com/yourusername/HelloMaven.git',
                branch: 'main'
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }
        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }
        stage('Archive') {
            steps {
                archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
            }
        }
    }
}
```

4. Run the Jenkins Job:

- Click “**Build Now**” and monitor the console output. Ensure that the build is successful and that test reports are generated.

B. Azure DevOps Build Pipeline Setup

1. Create a New Pipeline:

- Log in to your Azure DevOps project. ○ Navigate to “**Pipelines**” and click “**New pipeline**”.
- Select your repository source (GitHub or Azure Repos) and choose your repository.

2. Define Your YAML Pipeline:

Use the following sample YAML for a Maven project:

trigger:

- main

pool:

vmImage: 'ubuntu-latest'

steps:

- task: Maven@3

inputs:

mavenPomFile: 'pom.xml'

goals: 'clean package'

- task: PublishTestResults@2

inputs:

testResultsFiles: '**/target/surefire-reports/TEST-*.xml'

mergeTestResults: true

testRunTitle: 'Maven Unit Test Results'

3. Run the Pipeline and Verify Test Reports:

- Commit and run the pipeline.
- Navigate to the “**Tests**” tab to view the summary of executed tests

Step 3: Artifact Management

1. Artifact Archiving in Jenkins:

- Ensure your Jenkins job archives the artifact (JAR file) using the “**Archive the artifacts**” post-build action.

2. Artifact in Azure Pipelines:

- The build task produces an artifact that can be downloaded or referenced by subsequent release pipelines.

Step 4: Deployment Automation with Ansible and Azure Release Pipeline

A. Using Ansible for Deployment

1. Write an Ansible Playbook:

Create a file named deploy.yml:

```
---
- name: Deploy Maven Artifact
  hosts: deployment
  become: yes
  tasks:
  - name: Copy the artifact to the target directory
    copy:
      src: "/var/lib/jenkins/workspace/HelloMaven-CI/target/HelloMaven-1.0-SNA
SHOT.jar"
      dest: "/opt/deployment/HelloMaven.jar"
o
Adjust paths according to your environment.
```

2. Configure Your Ansible Inventory:

Create or update your hosts.ini file:

- [deployment]
- target-server ansible_host=your.server.ip ansible_user=yourusername
- For a local deployment, you can use:
- [deployment]
- localhost ansible_connection=local

3. Integrate Ansible into Your Jenkins/Azure Pipeline:

- Add a post-build (or post-release) step to execute the Ansible playbook:
- `ansible-playbook -i /path/to/hosts.ini /path/to/deploy.yml`

B. Using Azure Release Pipeline to Deploy to Azure App Services

1. Create a New Release Pipeline in Azure DevOps:

- Navigate to “**Pipelines > Releases**”.
- Click “**New pipeline**” and select an empty job.

2. Link Your Build Artifact:

- Click “**Add an artifact**” and select your build pipeline as the source.

3. Add a Deployment Stage for Azure App Service:

- Create a new stage (e.g., Production).
- Add the “**Azure App Service Deploy**” task.
- Configure the task with your Azure subscription, target App Service, and the path to the artifact.

4. Integrate Key Vault (Optional for Managing Secrets):

- Create and link a Variable Group that pulls secrets from Azure Key Vault (refer to previous experiments).
- Reference these secrets in your deployment tasks (e.g., connection strings).

5. Enable Continuous Deployment:

- In the release pipeline’s triggers, enable continuous deployment so that a new release is automatically created when a new artifact is available.

6. Run the Release Pipeline and Verify Deployment:

- Trigger the release pipeline (either manually or automatically).
- Verify that the application is deployed to your target environment (e.g., by browsing to the Azure App Service URL or checking the deployment directory on the target server).

Step 5: End-to-End Pipeline Demonstration

1. Trigger a Complete Run:

- Make a change in your code repository (e.g., modify a welcome message in your Java application) and commit it.
- This should trigger the CI pipeline (Jenkins or Azure Pipelines), which builds, tests, and archives the artifact.
- The artifact triggers the CD process (via Ansible or Azure Release), and the application is deployed automatically.

2. Verify the Entire Workflow:

- Check the CI pipeline output (build success and test results).
- Confirm that the artifact is archived.
- Review the CD pipeline logs (deployment success, any post-deployment notifications, etc.).
- Optionally, log into the target environment and verify that the new version of your application is running.

Step 6: Discussion on Best Practices and Q&A

Best Practices:

- **Pipeline as Code:**

Use YAML (or a Jenkinsfile) to define your build and release pipelines. This allows you to version control your pipeline configuration alongside your code.

- **Automate Everything:**

Automate code checkout, builds, tests, artifact archiving, and deployments. Reduce manual interventions to minimize human error.

- **Idempotence:**

Ensure that your deployment scripts (whether Ansible playbooks or release tasks) are idempotent—running them multiple times produces the same result.

- **Secure Secrets Management:**

Use Azure Key Vault (or a similar tool) to securely store sensitive data (e.g., API keys, connection strings) and reference these values in your pipelines.

- **Monitoring and Logging:**

Integrate logging and monitoring into your pipeline. Review test reports, deployment logs, and set up notifications for build failures.

- **Modular and Scalable Design:**

Break down your pipeline into clear stages (checkout, build, test, deploy) and design it to handle multi-environment deployments (development, staging, production).

- **Continuous Improvement:**

Regularly review and refine your pipeline. Use metrics and feedback to optimize build times,